KNOW GO

"A great deep dive into the latest Go features"

—Raina Audley-Miller

JOHN ARUNDEL

Contents

Praise for ‘Know Go’

Everything I wanted to know about generics in Go, beautifully explained!
—Pavel Anni

I really loved reading this book. I found the idea of generics scary at first, but now I’m very comfortable with it thanks to John’s simple yet complete way of teaching.
—Shivdas Patil

It’s taken me from being apprehensive about learning something new to being excited about the possibilities that generics brings.
—Dan Macklin

Well written: the explanations and examples are clear and easy to understand.
—Pedro Sandoval

One day’s exposure to mountains is better than a cartload of books.
—John Muir

Introduction

> *The more I use Go, the more I think generics would be a useless misfeature. Why do so many people think they need them?*
>
> —Aram Hăvărneanu

Hello, welcome to the book, and welcome to the world of generic programming in Go! I'm looking forward to exploring it with you.

About the book

This book is about *generics* in Go (among other things). We'll talk about exactly what that means in more detail later on, but, briefly, it's about defining (and using) generic functions and generic types.

About generics

First, what's a generic function? It's one that doesn't specify the types of all its parameters in advance. Instead, some types are represented by placeholders, called *type parameters*.

Generic *types* have the same property: some or all of the types involved aren't specified exactly. For example, we might want to define a slice of elements of some unspecified

type, or a struct with some field of a type that will be known later.

Like many things in programming, generics sounds complicated at first, but once you get your head round it, it's actually quite straightforward. In this book, we'll work together through the steps necessary to understand what generic functions and types are, why they're useful, how they work in Go, and what fun and interesting things we can do with them.

Who is the book for?

This book is for people who are new to the generics and iterators features in Go and want to know what they are, how to use them, and what they should do differently now that Go has them. If you have some experience using Go prior to the introduction of these features, and you just want to know what's new, you'll find everything you need to know right here.

If you're used to using generics and iterators in other languages, such as Java or C++, and you'd like to know how that experience will translate to Go, this book is also for you. If you've considered using Go in the past but decided against it for one reason or another, maybe the latest changes will tip the balance for you. This book will help you decide whether you'll be able to do what you want to do with Go.

And whether you have any experience with Go or not, you may be worried that these recent changes add unnecessary complexity to the language and will make it harder for you to understand, or even write, programs. This book is for you too! I hope you'll find that generic programming in Go isn't as difficult or complicated as it might sound. In fact, it's extremely straightforward, when we approach it the right way.

What should I read first?

If you're completely new to Go, or even to programming, I recommend you read my previous book, For the Love of Go, first. It'll give you a good grounding in the basics of Go, which will help you understand the material in *this* book more easily:

What will I learn from this book?

By reading through this book and completing the exercises, you'll learn:

- What we mean by *generic programming* in general, and specifically how that applies to Go
- What *type parameters* are, and how they differ from interfaces
- How to declare and write *generic functions*, and when that's necessary (and when it's not)
- How generic functions and types are implemented in Go, and how that affects the way we write programs

- How to define and use *constraints* on type parameters, and what constraints are provided in standard library packages and the Go language itself
- How to write *type element* constraints and *type approximations*
- What *operations* are allowed on parameterised types, and how to choose the right constraints for them
- How to define *generic types*, such as maps, slices, and structs, and how to write *methods* on such types.
- How to write *generic functions*, such as map, reduce, and filter operations on slices, and how to combine *first-class functions* with generics in a useful way.
- How generics support enables us to create some interesting *data structures* such as sets, trees, graphs, heaps, and queues.
- What *iterators* are, and how to create and use them, along with the new iterator APIs in the standard library.

Completing the challenges

You can learn a lot by just reading, of course, but you'll learn much more by taking on the many interactive code challenges throughout the book, and solving them.

How do I install the Go tools?

If a suitable version of Go isn't yet available as a package in your operating system distribution, you can build it from source or download a suitable binary package from the Go website directly:

- https://go.dev/learn/

Where are the code examples?

There are lots of puzzles for you to solve throughout the book, each designed to help you test your understanding of the concepts you've just learned. If you run into trouble, or just want to check your code, each exercise is accompanied by a complete sample solution, with tests.

All these solutions are also available in a public GitHub repo here:

- https://github.com/bitfield/know-go

Each exercise in the book is accompanied by a link to an example solution in the GitHub repo.

1. Interfaces

> *I went to a general store, but they wouldn't let me buy anything specific.*
>
> —Steven Wright

Programming with types

Still with me? Great. Let's lay the groundwork a little by talking about *types*.

Specific programming

I'm sure you know that Go has data types: numbers, strings, and so on. Every variable and value in Go has some type, whether it's a built-in type such as `int`, or a user-defined type such as a struct.

Go keeps track of these types, and you'll be well aware that it won't let you get away with any type mismatches, such as trying to assign an `int` value to a `float64` variable.

And you've probably written functions in Go that take some specific type of parameter, such as a string. If we tried to pass a value of a different type, Go would complain.

So that's *specific* programming, if you like: writing functions that take parameters of some specific type. And that's the kind of programming you're probably used to doing

in Go.

Generic programming

What would *generic* programming be, then? It would have to be writing functions that can take either *any* type of parameter, or, more usefully, a *set* of possible types.

The generic equivalent of our `PrintString` function, for example, might be able to print not just a string, but *any* type of value. What would that look like? What kind of parameter type would we declare?

It's tricky, because we have to put *something* in the function's parameter list, and we simply don't know what to write there yet. We will learn how generics solves this problem in a moment, but first let's look at some other ways we could have achieved the same goal.

Interface types

Go has always had a limited kind of support for functions that can take an argument of more than one specific type, using *interfaces*.

You might have encountered interface types like `io.Writer`, for example. Here's a function that declares a parameter `w` of type `io.Writer`:

```
func PrintTo(w io.Writer, msg string) {
    fmt.Fprintln(w, msg)
}
```

Here we don't know what the precise type of the argument `w` will be at run time (its *dynamic* type, we say), but we (and Go) can at least say something *about* it. We can say that it must implement the interface `io.Writer`.

What does it mean to implement an interface? Well, we can look at the interface definition for clues:

```
type Writer interface {
    Write(p []byte) (n int, err error)
}
```

What this is saying is that to be an `io.Writer`—to *implement* `io.Writer`—is to have a particular set of methods. In this case, just one method, `Write`, with a particular signature (it must take a `[]byte` parameter and return `int` and `error`).

This means that more than one type can implement `io.Writer`. In fact, any type that has a suitable `Write` method implements it automatically.

You don't even need to explicitly declare that your type implements a certain interface. If you have the right set of methods, you *implicitly* implement any interface that specifies those methods.

For example, we can define some struct type of our own, and give it a `Write` method that does nothing at all:

```
type MyWriter struct {}

func (MyWriter) Write([]byte) (int, error) {
    return 0, nil
}
```

We now know that the presence of this `Write` method implicitly makes our struct type an `io.Writer`. So we could pass an instance of `MyWriter` to any function that expects an `io.Writer` parameter, for example.

Interface parameters

The `MyWriter` type may not be very useful in practice, since it doesn't do anything. Nonetheless, any value of type `MyWriter` is a valid `io.Writer`, because it has the required `Write` method.

It can have *other* methods, too, but Go doesn't care about that when it's deciding whether or not a `MyWriter` is an `io.Writer`. It just needs to see a `Write` method with the correct signature.

This means that we can pass an instance of `MyWriter` to `PrintTo`, for example:

```
PrintTo(MyWriter{}, "Hello, world!")
```

If we tried to pass a value of some other type that *doesn't* satisfy the interface, we feel like it shouldn't work:

```
type BogusWriter struct{}

PrintTo(BogusWriter{}, "This won't compile!")
```

And indeed, we get this error:

```
cannot use BogusWriter{} (type BogusWriter) as type io.Writer
in argument to PrintTo:
    BogusWriter does not implement io.Writer
    (missing Write method)
```

That's fair enough. A function wouldn't declare a parameter of type `io.Writer` unless

it knew it needed to *call* `Write` on that value. By *accepting* that interface type, it's saying something about what it plans to do with the parameter: write to it!

Go can tell in advance that this won't work with a `BogusWriter`, because it doesn't have any such method. So it won't let us pass a `BogusWriter` where an `io.Writer` is expected.

Polymorphism

What's the point of all this, though? Why not just define the `PrintTo` function to take a `MyWriter` parameter, for example? That is to say, some *concrete* (non-interface) type?

Interfaces make code flexible

Well, you already know the answer to that: because *more than one* concrete type can be an `io.Writer`. There are many such types in the standard library: for example, `*os.File` or `*bytes.Buffer`. A function that takes a `io.Writer` can work with any of these.

Now we can see why interfaces are so useful: they let us write very flexible functions. We don't have to write multiple versions of the function, like `PrintToFile`, `PrintToBuffer`, `PrintToBuilder`, and so on.

Instead, we can write one function that takes an interface parameter, `io.Writer`, and it'll work with any type that implements this interface. Indeed, it works with types that don't even exist yet! As long as it has a `Write` method, it'll be acceptable to our function.

The fancy computer science term for this is *polymorphism* ("many forms"). But it just means we can take "many types" of value as a parameter, providing they implement some interface (that is, some set of methods) that we specify.

Constraining parameters with interfaces

Interfaces in Go are a neat way of introducing some degree of polymorphism into our programs. When we don't care what type our parameter is, so long as we can call certain methods on it, we can use an interface to express that requirement.

It doesn't have to be a standard library interface, such as `io.Writer`; we can define any interface we want.

For example, suppose we're writing some function that takes a value and turns it into a string, by calling a `String` method on it. What sort of interface parameter could we take?

Well, we know we'll be calling `String` on the value, so it *must* have at least a `String` method. How can we express that requirement as an interface? Like this:

```
type Stringer interface {
    String() string
}
```

In other words, any type can be a `Stringer` so long as it has a `String` method. Then we can define our `Stringify` function to take a parameter of this interface type:

```
func Stringify(s Stringer) string {
    return s.String()
}
```

In fact, this interface already exists in the standard library (it's called `fmt.Stringer`), but you get the point. By declaring a function parameter of interface type, we can use the same code to handle multiple dynamic types.

Note that all we can require about a method using an interface is its name and signature (that is, what types it takes and returns). We can't specify anything about what that method actually *does*.

Indeed, it might do nothing at all, as we saw with the `MyWriter` type, and that's okay: it still implements the interface.

Limitations of method sets

This "method set" approach to constraining parameters is useful, but fairly limited. Suppose we want to write a function that adds two numbers. We might write something like this:

```
func AddNumbers(x, y int) int {
    return x + y
}
```

That's great for `int` values, but what about `float64`? Well, we'd have to write essentially the same function again, but this time with a different parameter and result type:

```
func AddFloats(x, y float64) float64 {
    return x + y
}
```

The actual logic (`x + y`) is exactly the same in both cases, so the type system is hurting us more than it's helping us here.

Indeed, we'd also have to write `AddInt64s`, `AddInt32s`, `AddUints`, and so on, and they'd

all consist of the same code. This is boring, and it's not the kind of thing that we became programmers to do.

So we need to think of something else. Maybe interfaces can come to our rescue?

Let's try. Suppose we change `AddNumbers` to take a pair of parameters of some interface type, instead of a concrete type like `int` or `float64`.

What interface could we use? In other words, what methods would we need to specify that the parameter type must implement?

Well, here's where we run into a limitation of interfaces defined by method sets. Actually, `int` *has* no methods, and nor do any of the other built-in types! So there's no method set we can specify that would be implemented by `int`, `float64`, and friends.

We could still define *some* interface: for example, we could require an `Add` method, and then we could define struct types with such a method, and pass them to `AddNumber`. Great. But it wouldn't allow us to use any of Go's *built-in* number types, and that would be a most inconvenient limitation.

The empty interface: `any`

Here's another idea. What about the *empty* interface, named `any`? That specifies no methods at all, so literally every concrete type implements it.

Could we use `any` as the type of our parameters? Since that would allow us to pass arguments of any type at all, we might rename our function `AddAnything`:

```
// invalid
func AddAnything(x, y any) any {
    return x + y
}
```

Unfortunately, this doesn't compile:

```
invalid operation: x + y (operator + not defined on interface)
```

The problem here is what we tried to *do* with the parameters: that is, add them. To do that, we used the `+` operator, and that's not allowed here.

Why not? Because we said, in effect, that `x` and `y` can be *any* type, and not every type works with the `+` operator.

If `x` and `y` were instances of some kind of struct, for example, what would it even *mean* to add them together? There's no way to know, so Go plays it safe by disallowing the `+` operation altogether.

And there's another, subtler problem here, too. We presumably need `x` and `y` to be the *same* concrete type, whatever it is. But because they're both declared as `any`, we can call

this function with *different* concrete types for `x` and `y`, which almost certainly wouldn't make sense.

Type assertions and switches

You probably know that we can write a *type assertion* in Go, to detect what concrete type an interface value contains. And there's a more elaborate construct called a *type switch*, which lets us detect a whole set of possible types, like this:

```
switch v := x.(type) {
case int:
    return v + y
case float64:
    return v + y
case ...
```

Using a `switch` statement like this, we can list all the concrete types that we know *do* support the `+` operator, and use it with each of them.

This seems promising at first, but really we're just right back where we started! We wanted to avoid writing a separate, essentially identical version of the `Add` function for each concrete type, but here we are doing basically just that. So an interface is no use here.

In practice, we don't often need to write functions like `AddAnything`, which is just as well. But this is an awkward limitation of Go, or so it would seem: it makes it difficult for us to write general-purpose packages, among other things.

Look at the `math` package in the standard library, for example. It provides lots of useful utility functions such as `Pow`, `Abs`, and `Max`... but only on `float64` values.

If you want to use those functions with some other type, you'll have to explicitly convert it to `float64` on the way in, and back to your preferred type on the way out. That's just lame.

Go, meet generics

This isn't full generic programming, then, in the way that we now understand the term. We can't write the equivalent of an `AddAnything` function using just method-based interfaces, even the empty interface.

Or rather, we can write functions that *take* values of any type: we just can't *do* anything useful with those values, like add them together.

How it started

At least, that was true until recently, but now there *is* a way to do this kind of thing. We can use Go generics!

We'll see in the next chapter what that actually involves, but let's take a very brief trip through the history of Go to see how we got where we are today.

> *Go was released on November 10, 2009. Less than 24 hours later we saw the first comment about generics.*
>
> —Ian Lance Taylor, "Why Generics?"

Go was deliberately designed to be a very simple language, and also to make it easy to compile and build Go programs very fast, without using a lot of resources. That means it doesn't have everything, and generics is one of the features Go didn't have at launch.

Why didn't the designers just add generics later, then? Well, there are a couple of compelling reasons.

How it's going

Go was intended to be quick to learn, without a lot of syntax and keywords to master before you can be productive with the language. Every new thing you add to it is something else that beginners will have to learn.

The Go team also puts a great value on *backwards compatibility*: that is to say, no breaking changes can be introduced to the language. So if you introduce some new syntax, it has to be done in a way that doesn't conflict with any possible existing programs. That's hard!

Various proposals for generics in Go have been made over the years, in fact, but most of them fell at one or another of these hurdles. Some involved an unacceptable hit to compiler performance, or to runtime performance; others introduced too much complexity or weren't backwards compatible with existing code.

That's why it took about ten years for generics to finally land in Go. What we have, after all that thinking and arguing, is actually a very nice design. Like Go itself, it does a lot with a little.

With the absolute minimum of new syntax, the Go team have opened up a whole new world of programming, and enabled us to write new kinds of programs in Go. It'll take a while for the consequences of all this to feed through into the mainstream, and for most people, even for experienced Gophers, generics are something very new.

In the next chapter, we'll talk about exactly what it is that's been added to Go, and start writing some generic programs of our own.

2. Type parameters

> *I learned very early the difference between knowing the name of something, and knowing something.*
>
> —Richard Feynman, "The World From Another Point of View"

Now that we've described what generic programming is, and how it came to Go, it's time to look at exactly what Go's generics support involves, and what we can do with it.

Generic functions

And now that's easier to explain: it means we can write functions like `PrintAnything` and `AddAnything`!

Introducing T, the arbitrary type

To be precise, we can write *generic functions* that take parameters not of some specific named type, but of some arbitrary type that we don't have to specify in advance. Let's call it T, for "type".

When the function is actually *called* in our program, T will be some specific type, such

as `int`. But we don't want to have to specify that in advance when we're writing the function, so we're just going to use T as a placeholder for whatever the type ends up being.

In fact, there might be more than one T in our finished program. For example, we might call our generic function with a `float64` value, in which case T will be `float64`. But elsewhere we might call the same function with a `string` value, in which case T will be `string`.

This T placeholder is called a *type parameter*. A generic function in Go, then, is a function that takes a type parameter. So what does that look like?

Let's take the simplest imaginable case first. We'll write a `PrintAnything` function with a type parameter T, where T is a placeholder for any type:

```
func PrintAnything[T any](v T) {
```

Type parameters

What does this mean? For any type T, `PrintAnything[T]` takes a T parameter (that is, a parameter whose type is T), and returns nothing. The function signature just says that in Go instead of English.

While `v` is just an ordinary parameter, of some unspecified type, T is different. T is a new kind of parameter in Go: a *type parameter*.

We say that `PrintAnything` is a *parameterised* function, that is, a generic function *on* some type T. For short, we usually just talk about `PrintAnything[T]`, pronounced "`PrintAnything` *of* T".

Instantiation

What type *is* T, specifically? It depends what we decide to pass to the function when it's called.

Suppose we want to pass it an `int` value, for example:

```
var x int = 5
PrintAnything(x)
// Output:
// 5
```

Go is smart enough to know that `x` is an `int`, and therefore it needs to compile (and call) a version of `PrintAnything[T]` where T is `int`.

This is called *instantiating* the function, as in "creating an instance of it". In effect, the generic `PrintAnything` function is like a kind of template, and when we call it with

some specific type, we create a specific *instance* of the function that takes that type—for example, `int`.

What if we have another call to `PrintAnything[T]` somewhere else in the program, and this time T is a different type, such as `string`? Well, that's okay. Go will produce another version of `PrintAnything`, this time one that takes a string argument.

In most cases, as in these examples, Go can *infer* the type of T from the value that's supplied at the site of the function call. When that isn't possible, Go will let you know.

For example, suppose we have a generic function `Something[T any]` that *returns* a value of T. And suppose we call it like this:

```
x := Something()
```

What is T here? We don't know, and nor does Go, so it complains:

```
cannot infer T
```

All is not lost, however. We can specify which particular T we want in this case by putting it inside square brackets after the function name:

```
x := Something[int]()
```

It's always okay to *explicitly instantiate* a parameterised function or type in this way. But you only *have* to do it when Go can't automatically infer the required type, which isn't all that often.

Stencilling

This approach to implementing generics is sometimes called *stencilling*, which is a rather apt name. You can imagine Go spray-painting a bunch of similar versions of the function, differing only in the type of the parameter they take.

We could have done the same thing ourselves using the existing *code generation* machinery in Go, and indeed many people did do exactly that before the introduction of generics.

This makes for efficient machine code, because there's no indirection (unlike with interface values). We don't need type assertions, because each different implementation of `PrintAnything` knows exactly what concrete type it's getting.

This isn't a particularly compelling example of generics in action, though, because it was already possible to write `PrintAnything` using a method-set interface: `any`. We could just pass the argument straight to `fmt.Println`, which also takes `any`.

Getting started

Let's look at a slightly more interesting, though still rather contrived, example.

An `Identity` function

Suppose we want to write a function called `Identity` that simply returns whatever value you pass it. (I know it doesn't sound very useful, but bear with me.) How could we write such a function in Go, without having to implement a separate version for each possible type of value?

This is where we start to go beyond the limits of interfaces. Using `any`, for example, we'd have to write something like:

```
func Identity(v any) any {
    return v
}
```

This works, but it isn't really satisfactory. As we saw with `AddAnything` in the previous chapter, we don't have any way to tell Go that the function's parameter and its result must be the *same* concrete type, whatever it is. And clearly that needs to be the case here: if we pass this function an `int`, we expect to get an `int` back.

Now we know how to specify that requirement, using a type parameter:

```
func Identity[T any](v T) T {
    return v
}
```

Notice that, although it looks similar to the non-generic version, there's an important difference. Whatever T turns out to be (and it can be any type), *both* the parameter and the function's result must be of that type.

Instantiating `Identity`

Suppose we call this function somewhere in our program with a string argument, then:

```
fmt.Println(Identity("Hello"))
// Output:
// Hello
```

You now know how this works. Under the hood, Go instantiates a version of `Identity` that takes a string parameter and returns a string result. This is just a plain, ordinary Go function that we could have written ourselves, or generated mechanically.

The point is, of course, that *we* don't need to supply a separate version of `Identity` for each concrete type that we want to use. Instead, we just write it once for some arbitrary type T, and Go will automatically generate a version of `Identity` for each type that's actually used in our program.

Exercise: Hello, generics

Now it's over to you to write your first generic function in Go! Let's work through it together, step by step.

First of all, make sure you've checked out a copy of the GitHub repo for this book:

- https://github.com/bitfield/know-go

Open the `exercises/print` folder in your code editor and take a look at the `print_test.go` file.

You'll find this test:

```
func TestPrintAnythingTo_PrintsToGivenWriter(t *testing.T) {
    t.Parallel()
    buf := new(bytes.Buffer)
    print.PrintAnythingTo(buf, "Hello, world")
    want := "Hello, world\n"
    got := buf.String()
    if want != got {
        t.Errorf("want %q, got %q", want, got)
    }
}
```

(Listing `exercises/print`)

Running the test

Run the test using your editor, or the `go test` command. You'll see that right now the test doesn't compile, because the required function doesn't exist:

```
undefined: print.PrintAnythingTo
```

Remember, you'll need at least Go version 1.18 to be able to use generics, including running this test and implementing the function to make it pass. That's because we're using some new syntax that doesn't exist in Go version 1.17 and earlier.

If you try to compile this code with an older version of Go that doesn't support generics, you'll get a rather confusing additional error message:

```
type string is not an expression
```

So if you see this, you need to upgrade your Go. Follow the instructions in the introduction to this book to do that. Now read on!

GOAL: Get the test passing!

HINT: To make this test even compile, you'll need to define a generic function in the `print` package named `PrintAnythingTo` that takes one parameter of type `io.Writer`, and another value that's of some unspecified type.

In other words, `PrintAnythingTo` has a type parameter we'll refer to as T, which can be any type, and it takes a parameter of this type T, just like `Identity`. But unlike `Identity`, it also takes another parameter, which is of type `io.Writer`.

To make the test *pass*, your function will need to write the supplied value to the supplied writer. It's up to you how to do this, but you might like to use `fmt.Fprintln`, like our `PrintTo` example in the previous chapter.

The necessary `go.mod` and `print.go` files are already set up for you. All you need to do is edit `print.go` and add the `PrintAnythingTo` function, then run the test again.

SOLUTION: Here's my suggested solution; it's fine if yours looks different, so long as it passes the test. It's just for comparison, or if you need a little extra help on this tricky first exercise.

```
package print

import (
    "fmt"
    "io"
)

func PrintAnythingTo[T any](w io.Writer, p T) {
    fmt.Fprintln(w, p)
}
```

(Listing `solutions/print`)

We use the `[T any]` syntax to say that `PrintAnything` is about some arbitrary type T, and it takes a parameter—`p`, the thing to print—of that type, whatever it actually turns out to be. In the test, as it happens, that's a string, but it *could* have been any type. If you like, you can add more tests that call `PrintAnything` with different types, such as `int` or `float64`.

Composite types

So far, we've figured out how to define a generic function that, for some type T, takes a parameter of type T:

```
func Identity[T any](v T) {
```

So is that it? Are we restricted to declaring only parameters of type T itself, or could we also take some *composite* type? That is, some type *involving* T, not just T itself? For example, a *slice* of T?

Slices of some arbitrary type

We could indeed. Suppose we wanted to write a function `Len` that returns the length of a given slice. And its parameter will always be a slice of *something*: that is, of some arbitrary element type. Let's call it E, for "element".

The signature of `Len`, then, might look something like this:

```
func Len[E any](s []E) int {
```

It's conventional, though not required, to capitalise the names of type parameters. Those names can be whatever you like, but again it's conventional to use a single letter: T for any type, E for an element type of a slice, and so on.

Other generic composite types

But we can also use a type parameter in other kinds of composite type. For example, we can write a generic function on a *channel* of some element type E:

```
func Drain[E any](ch <-chan E) {}
```

We could even write a *variadic* function: that is, a function that takes a variable number of arguments. In this case, it would take a variable number of channels of E:

```
func Merge[E any](chs ...<-chan E) <-chan E {
```

Actually implementing these functions is a topic for another book (stay tuned), but I'm sure you can see the possibilities.

Generic types

Generic functions are great, but we can do more. We can also write generic *types*. What does that mean?

We often deal with *collections* of values in Go. For example, consider this slice type:

```
type SliceOfInt []int
```

You already know that, just as an `int` variable can only hold `int` values, a `[]int` slice can only hold `int` elements. We wouldn't want to have to also define `SliceOfFloat`, `SliceOfString`, and so on.

Defining a generic slice type

Could we write a *generic* type definition that takes a type parameter, just like a generic function? For example, could we make a slice of any type?

Yes, we could:

```
type Bunch[E any] []E
```

Just as we could define a function such as `Len` that takes a slice of an arbitrary element type, we've now given a name to a new *type*: a slice of E, for some type E.

And just as with generic functions, a generic type is always *instantiated* on some specific type when it's used in a program. That is to say, a `Bunch[int]` will be a slice of `int`, a `Bunch[string]` will be a slice of strings, and so on.

Each of these is a distinct concrete type, as you'd expect, and we can write a `Bunch` literal by giving the type we want in square brackets:

```
b := Bunch[int]{1, 2, 3}
```

The elements all have the same type

It's very important to understand that, even though a `Bunch` is defined as a slice of E for any type E, any *particular* `Bunch` can't contain values of *different* types. All the elements of a `Bunch[T]` must be of type T, whatever it is.

For example, we can't create a `Bunch` of one type, and then try to append a value of a different type:

```
b := Bunch[int]{1, 2, 3}
b = append(b, "hello")
// cannot use "hello" (untyped string constant) as int value in
// argument to append
```

We can have a `Bunch[int]` *or* a `Bunch[string]` *or* a `Bunch` of elements of any other specific type. What we can't have is a `Bunch` of *mixed-type* elements.

It's easy to hear a term like "generic slice" and jump to the conclusion that it means "slice containing values of different types". But that's actually not the case.

Generic types need to be instantiated

There are, in a sense, *no generic types in Go*. I know that sounds crazy, but stick with me.

That is, you can *define* generic types like `Bunch[E]`, but to actually use them in your program, you need to *instantiate* them on some specific type, like `int`.

At that point, what you have is an ordinary Go slice of `int`, and it stands to reason that such a slice can only contain `int` elements.

So just because we used a type parameter, it doesn't mean we can create a single slice that contains elements of different types. What we can create are different *slice types*, such as `[]int`, or `[]string`, without having to specify their element type in advance.

One way to express this is to say that, while we can define generic types at compile time, there are no generic types *at run time*.

There's one partial exception to this, which you're already familiar with: interface types. We could always create a slice of `any` in Go, and populate it with elements of different dynamic types. But, for the reasons we've discussed, this isn't the same thing as a truly generic type.

Exercise: Group therapy

Over to you again now, to try your hand at the `group` exercise. This time, you'll need to define a generic slice type `Group[E]` to pass the following test:

```
func TestGroupContainsWhatIsAppendedToIt(t *testing.T) {
    t.Parallel()
    got := group.Group[string]{}
    got = append(got, "hello")
    got = append(got, "world")
    want := group.Group[string]{"hello", "world"}
    if !slices.Equal(want, got) {
        t.Errorf("want %v, got %v", want, got)
    }
}
```

(Listing `exercises/group`)

GOAL: Get the test passing!

HINT: Well, a "group" is a lot like a "bunch", and I don't think you'll need any more hints than that. There's no trick here: it's just a confidence builder to get you used to defining generic types.

SOLUTION: And here's what that looks like, just as we saw before with the `Bunch` type:

```
type Group[E any] []E
```

(Listing `solutions/group`)

I told you it was easy!

Generic function types

You probably know that *functions are values* in Go. That is, you can pass a function as an argument to another function, you can *return* a function from a function, and you can declare variables or struct fields of a particular function type.

In other words, just as Go values can be integers, floats, or strings, they can also be functions. For example, a function that takes an `int` parameter and returns `bool` would have the type `func(int) bool`. This is a very handy feature of Go, and we use it a lot.

Generic functions as values

So what if that function were generic? For example, the `Identity[T]` function in our earlier example. How would we declare a variable to which we could assign the function `Identity[T]`? What would be the type of such a variable?

Well, you now know that there's actually no such function as `Identity[T]`, only one or more *instantiations* of that function on a specific type, such as `string`. As we've seen, the `Identity[T]` function definition is just a kind of "stencil" that Go uses to produce *actual* functions.

So there *is* such a function as `Identity[string]`, and accordingly we can use it as a value. For example, we can assign it to a variable:

```
f := Identity[string]
```

The type is always instantiated

What is the type of the variable `f`, then? Well, it's whatever the type of `Identity[string]` is. Here's the signature of the generic `Identity` function again:

```
func Identity[T any](v T) T {
```

So if T is `string` in this case, then we feel the type of `Identity[string]` should be:

```
func(string) string
```

Let's ask Go itself. Conveniently, the `fmt` package can report the type of a value, using the `%T` verb. So we'll see what type it thinks `f` is in this case:

```
f := Identity[string]
fmt.Printf("%T\n", f)
// Output:
// func(string) string
```

How straightforward!

There are no generic functions

Just as with generic types, then, *there are no generic functions in Go*, if you want to be a smart-alec about it (and I do).

Any generic functions you may write will in fact be instantiated on some specific type at compile time, and they will then just be plain old functions, like `func(string) string`. If we wanted to give such a specific function type a name, we could:

```
type Stringulator func(string) string
```

Could we define a *generic function type*, though? That is, give a name to the type of a generic function on some arbitrary type T.

For example, could we write something like:

```
type idFunc func[T any](T) T
// syntax error: function type must have no type parameters
```

Nope. If you think about it, how could Go compile this? It couldn't, because *all type arguments must be known at compile time*.

What we *can* write is this:

```
type idFunc[T any] func(T) T
```

This is fine, because it's a *generic type*, just like the ones we've already seen. For some T, we're saying, an `idFunc[T]` is a function that takes a T and returns a T.

If this is ever instantiated in our program, it will become some specific type like `func(int) int`. If not, Go can safely ignore it. Either way, no unknown types are involved.

Generic types as function parameters

We can create both generic functions and generic types, as we've seen. So an interesting question occurs: could we write a generic function that takes a *parameter* of a generic type?

The answer is absolutely yes:

```
func PrintBunch[E any](v Bunch[E]) {
    fmt.Println(v)
}

func main() {
    b := Bunch[int]{1, 2, 3}
    PrintBunch(b)
    // Output:
    // [1, 2, 3]
}
```

This is pretty exciting stuff! But there's a limit to what we can do with generic functions that take literally *any* type. We'll see what that limit is in a moment, but first, let's do a little more confidence building.

Exercise: Lengthy proceedings

Now it's your turn to write a generic function on a generic type, to solve the `length` exercise.

```
func TestLenOfSliceIs2WhenItContains2Elements(t *testing.T) {
    t.Parallel()
    s := []int{1, 2}
    want := 2
    got := length.Len(s)
    if want != got {
        t.Errorf("Len(%v): want %d, got %d", s, want, got)
    }
}
```

(Listing `exercises/length`)

GOAL: Your task is to implement a generic function `Len[E]` that returns the length of a given slice. As usual, the test will tell you when you've got it right!

HINT: Actually, the *implementation* is easy, isn't it? We don't need to write code to figure out the length of some slice; there's a built-in function for that (see if you can guess which one).

The tricky bit, if it is tricky, might be writing the *signature* of the `Len` function. But it's not really any more complicated than what we've seen already.

For example, the `PrintBunch` function takes a `Bunch[E]` for some arbitrary element type E. Well, so does this one! Can you see what to do?

SOLUTION: Here's one possible answer:

```
func Len[E any](s []E) int {
    return len(s)
}
```

(Listing `solutions/length`)

Constraining type parameters

We saw in the previous chapter that one of the limitations of interface types in Go is that we can't use them with operators such as `+`. Remember our `AddAnything` example?

We can't add `any` to `any`

You might be wondering if the same kind of limitation applies to functions parameterised by `T any`, and indeed it does. If we try to write a generic `AddAnything[T any]`, for example, that doesn't work:

```
func AddAnything[T any](x, y T) T {
    return x + y
}
// invalid operation: operator + not defined on x (variable
// of type T constrained by any)
```

Go is always right, but it can sometimes express itself in a rather terse way, so let's unpack that error message a little.

It's complaining about this line:

```
return x + y
```

And it's saying:

```
operator + not defined on x
```

To put it another way, Go has no way to guarantee that whatever specific type T happens to be when the function is instantiated, that type will work with the + operator.

It *might*, but then again, it might not, because:

```
variable of type T constrained by any
```

Not every type is "addable"

In other words, because `x` can be *any* type, there's no way to guarantee that, when the program runs, `x` will be one of the types that supports the + operator. So this is just the same kind of problem as when we tried to add together two `any` values.

If `x` and `y` were some struct type, for example, that certainly wouldn't work: structs don't support +, because it's not meaningful to add two structs together. The + operator isn't *defined* on structs, we say.

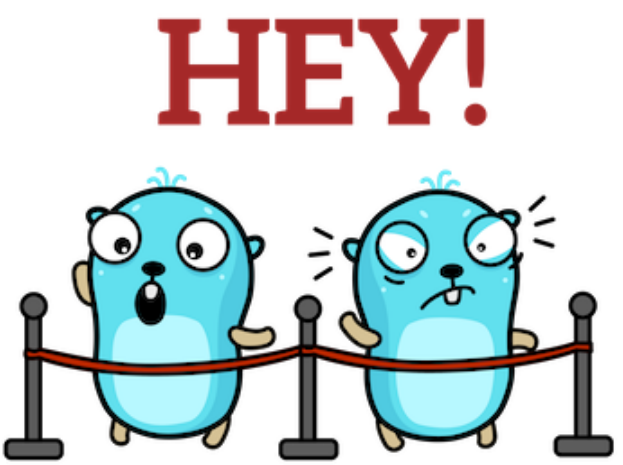
Go is telling us that we can't write the expression `x + y` with values of literally *any* type T: that's too broad a range of possible types.

It might be that, in a given program, we only ever instantiate `AddAnything` on types that *do* happen to support +, such as `int` or `string`. But that's not good enough for Go: we need to *guarantee* that it can't be instantiated on some inappropriate type.

To do this, we need to *constrain* T a little more. That is, to restrict the allowed possibilities for T to only those types that support the operator we want to use. And we'll see how to do that in the next chapter.

3. Constraints

> *Design is the beauty of turning constraints into advantages.*
>
> —Aza Raskin



We saw in the previous chapter that when we're writing generic functions that take any type, the range of things we can *do* with values of that type is necessarily rather limited. For example, we can't add them together. For that, we'd need to be able to prove to Go that they're one of the types that support the + operator.

Method set constraints

It's the same with interfaces, as we discussed in the first chapter. The empty interface, `any`, is implemented by every type, and so knowing that something implements `any` tells you nothing distinctive about it.

Limitations of the `any` constraint

Similarly, in a generic function parameterised by some type T, constraining T to `any` doesn't give Go any information about it. So it has no way to guarantee that a given operator, such as +, will work with values of T.

A Go proverb says:

> *The bigger the interface, the weaker the abstraction.*
>
> —https://go-proverbs.github.io/

And the same is true of constraints. The broader the constraint, and thus the more types it allows, the less we can guarantee about what operations we can do on them.

There *are* a few things we can do with `any` values, as you already know, because we've done them. For example, we can declare variables of that type, we can assign values to them, we can return them from functions, and so on.

But we can't really do a whole lot of *computation* with them, because we can't use operators like `+` or `-`. So in order to be able to do something useful with values of T, such as adding them, we need more restrictive constraints.

What kinds of constraints *could* there be on T? Let's examine the possibilities.

Basic interfaces

One kind of constraint that we're already familiar with in Go is an *interface*. In fact, all constraints are interfaces of a kind, but let's use the term *basic* interface here to avoid any confusion. A basic interface, we'll say, is one that contains only method elements.

For example, the `fmt.Stringer` interface we saw in the opening chapter:

```
type Stringer interface {
    String() string
}
```

We've seen that we can write an ordinary, non-generic function that takes a parameter of type `Stringer`. And we can also use this interface as a type constraint for a generic function.

For example, we could write a generic function parameterised by some type T, but this time T can't be just any type. Instead, we'll say that whatever T turns out to be, it must implement the `fmt.Stringer` interface:

```
func Stringify[T fmt.Stringer](s T) string {
    return s.String()
}
```

This is clear enough, and it works the same way as the generic functions we've already written. The only new thing is that we used the constraint `Stringer` instead of `any`. Now when we actually call this function in a program, we're only allowed to pass it arguments that implement `Stringer`.

What would happen, then, if we tried to call `Stringify` with an argument that *doesn't* implement `Stringer`? We feel instinctively that this shouldn't work, and it doesn't:

```
fmt.Println(Stringify(1))
// int does not implement Stringer (missing method String)
```

That makes sense. It's just the same as if we wrote an ordinary, non-generic function that took a parameter of type `Stringer`, as we did in the first chapter.

There's no advantage to writing a generic function in this case, since we can use this interface type directly in an ordinary function. All the same, a basic interface—one defined by a set of methods—is a valid constraint for type parameters, and we can use it that way if we want to.

Exercise: Stringy beans

Flex your generics muscles a little now, by writing a generic function constrained by `fmt.Stringer` to solve the `stringy` exercise.

```
type greeting struct{}

func (greeting) String() string {
    return "Howdy!"
}

func TestStringifyTo_PrintsToSuppliedWriter(t *testing.T) {
    t.Parallel()
    buf := &bytes.Buffer{}
    stringy.StringifyTo[greeting](buf, greeting{})
    want := "Howdy!\n"
    got := buf.String()
    if want != got {
        t.Errorf("want %q, got %q", want, got)
    }
}
```

(Listing `exercises/stringy`)

GOAL: Your job here is to write a generic function `StringifyTo[T]` that takes an `io.Writer` and a value of some arbitrary type constrained by `fmt.Stringer`, and prints the value to the writer.

HINT: This is a bit like the `PrintAnything` function we saw before, isn't it? Actually, it's a "print anything stringable" function. We already know what the constraint is

(`fmt.Stringer`), and the rest is straightforward.

SOLUTION: Here's a version that would work, for example:

```
func StringifyTo[T fmt.Stringer](w io.Writer, p T) {
    fmt.Fprintln(w, p.String())
}
```

(Listing `solutions/stringy`)

Strictly speaking, of course, we don't really need to call the `String` method: `fmt` already knows how to do that automagically. But if we just passed `p` directly, we wouldn't need the `Stringer` constraint, and we could use `any`... but what would be the fun in that?

Type set constraints

We've seen that one way an interface can specify an allowed range of types is by including a *method element*, such as `String() string`. That would be a basic interface, but now let's introduce another kind of interface. Instead of listing methods that the type must have, it directly specifies a set of types that are allowed.

Type elements

For example, suppose we wanted to write some generic function `Double` that multiplies a number by two, and we want a type constraint that allows only values of type `int`. We know that `int` has no methods, so we can't use any basic interface as a constraint. How can we write it, then?

Well, here's how:

```
type OnlyInt interface {
    int
}
```

Very straightforward! It looks just like a regular interface definition, except that instead of method elements, it contains a single *type element*, consisting of a named type. In this case, the named type is `int`.

Using a type set constraint

How would we use a constraint like this? Let's write `Double`, then:

```
func Double[T OnlyInt](v T) T {
    return v * 2
}
```

In other words, for some T that satisfies the constraint `OnlyInt`, `Double` takes a T parameter and returns a T result.

Note that we now have one answer to the sort of problem we encountered with `AddAnything`: how to enable the `*` operator (or any other arithmetic operator) in a parameterised function. Since T can only be `int` (thanks to the `OnlyInt` constraint), Go can guarantee that the `*` operator will work with T values.

It's not the complete answer, though, since there are other types that support `*` that *wouldn't* be allowed by this constraint. And in any case, if we were only going to support `int`, we could have just written an ordinary function that took an `int` parameter.

So we'll need to be able to expand the range of types allowed by our constraint a little, but not beyond the types that support `*`. How can we do that?

Unions

What types *can* satisfy the constraint `OnlyInt`? Well, only `int`! To broaden this range, we can create a constraint specifying more than one named type:

```
type Integer interface {
    int | int8 | int16 | int32 | int64
}
```

The types are separated by the pipe character, `|`. You can think of this as representing "or". In other words, a type will satisfy this constraint if it is `int` *or* `int8` *or*... you get the idea.

This kind of interface element is called a *union*. The type elements in a union can include any Go types, including interface types.

It can even include other constraints. In other words, we can *compose* new constraints from existing ones, like this:

```
type Float interface {
    float32 | float64
}

type Complex interface {
    complex64 | complex128
}
```

```
type Number interface {
    Integer | Float | Complex
}
```

We're saying that `Integer`, `Float`, and `Complex` are all unions of different built-in numeric types, but we're also creating a new constraint `Number`, which is a union of those three *interface* types we just defined. If it's an integer, a float, or a complex number, then it's a number!

The set of all allowed types

The *type set* of a constraint is the set of all types that satisfy it. The type set of the empty interface (`any`) is the set of all types, as you'd expect.

The type set of a union element (such as `Float` in the previous example) is the union of the type sets of all its terms.

In the `Float` example, which is the union of `float32 | float64`, its type set contains `float32`, `float64`, and no other types.

Intersections

You probably know that with a basic interface, a type must have *all* of the methods listed in order to implement the interface. And if the interface contains other interfaces, a type must implement *all* of those interfaces, not just one of them.

For example:

```
type ReaderStringer interface {
    io.Reader
    fmt.Stringer
}
```

If we were to write this as an *interface literal*, we would separate the methods with a semicolon instead of a newline, but the meaning is the same:

```
interface { io.Reader; fmt.Stringer }
```

To implement this interface, a type has to implement *both* `io.Reader` *and* `fmt.Stringer`. Just one or the other isn't good enough.

Each line of an interface definition like this, then, is treated as a distinct type element. The type set of the interface as a whole is the *intersection* of the type sets of all its elements. That is, only those types that all the elements have in common.

So putting interface elements on different lines has the effect of requiring a type to implement *all* those elements. We don't need this kind of interface very often, but we can imagine cases where it might be necessary.

Empty type sets

You might be wondering about what happens if we define an interface whose type set is completely empty. That is, if there are no types that can satisfy the constraint.

Well, that could happen with an intersection of two type sets that have *no* elements in common. For example:

```
type Unpossible interface {
    int
    string
}
```

Clearly no type can be both `int` and `string` at the same time! Or, to put it another way, this interface's type set is empty.

If we try to instantiate a function constrained by `Unpossible`, we'll find, naturally enough, that it can't be done:

```
cannot implement Unpossible (empty type set)
```

We probably wouldn't do this on purpose, since an unsatisfiable constraint doesn't seem that useful. But with more sophisticated interfaces, we might accidentally reduce the allowed type set to zero, and it's helpful to know what this error message means so that we can fix the problem.

Composite type literals

A *composite* type is one that's built up from other types. We saw some composite types in the previous chapter, such as `[]E`, which is a slice of some element type E.

But we're not restricted to defined types with names. We can also construct new types on the fly, using a *type literal*: that is, literally writing out the type definition as part of the interface.

A struct type literal

For example, this interface specifies a *struct* type literal:

```
type Pointish interface {
    struct{ X, Y int }
```

```
}
```

A type parameter with this constraint would allow any instance of such a struct. In other words, its type set contains exactly one type: `struct{ X, Y int }`.

Access to struct fields

While we can write a generic function constrained by some struct type such as `Pointish`, there are limitations on what that function can do with that type. One is that it can't access the struct's *fields*:

```
func GetX[T Pointish](p T) int {
    return p.X
}
// p.X undefined (type T has no field or method X)
```

In other words, we can't refer to a field on `p`, even though the function's constraint explicitly says that any `p` is guaranteed to be a struct with at least the field `X`. This is a limitation of the Go compiler that has not yet been overcome. Sorry about that.

Some limitations of type sets

An interface containing type elements can *only* be used as a constraint on a type parameter. It can't be used as the type of a variable or parameter declaration, like a basic interface can. That too is something that might change in the future, but this is where we are today.

Constraints versus basic interfaces

What exactly stops us from doing that, though? We already know that we can write functions that take ordinary parameters of some basic interface type such as `Stringer`. So what happens if we try to do the same with an interface containing type elements, such as `Number`?

Let's see:

```
func Double(p Number) Number {
// interface contains type constraints
```

This doesn't compile, for the reasons we've discussed. Some potential confusion arises from the fact that a basic interface can be used as both a regular interface type *and* a constraint on type parameters. But interfaces that contain type elements can only be used as constraints.

Constraints are not classes

If you have some experience with languages that have *classes* (hierarchies of types), then there's another thing that might trip you up with Go generics: constraints are not classes, and you can't instantiate a generic function or type on a constraint interface.

To illustrate, suppose we have some concrete types `Cow` and `Chicken`:

```
type Cow struct{ moo string }

type Chicken struct{ cluck string }
```

And suppose we define some interface `Animal` whose type set consists of `Cow` and `Chicken`:

```
type Animal interface {
    Cow | Chicken
}
```

So far, so good, and suppose we now define a generic type `Farm` as a slice of `T Animal`:

```
type Farm[T Animal] []T
```

Since we know the type set of `Animal` contains exactly `Cow` and `Chicken`, then either of those types can be used to instantiate `Farm`:

```
dairy := Farm[Cow]{}
poultry := Farm[Chicken]{}
```

What about `Animal` itself? Could we create a `Farm[Animal]`? No, because there's no such type as `Animal`. It's a type *constraint*, not a type, so this gives an error:

```
mixed := Farm[Animal]{}
// interface contains type constraints
```

And, as we've seen, we also couldn't use `Animal` as the type of some variable, or ordinary function parameter. Only basic interfaces can be used this way, not interfaces containing type elements.

Approximations

Let's return to our earlier definition of an interface `Integer`, consisting of a union of named types. Specifically, the built-in signed integer types:

```
type Integer interface {
    int | int8 | int16 | int32 | int64
}
```

We know that the type set of this interface contains all the types we've named. But what about defined types whose *underlying* type is one of the built-in types?

Limitations of named types

For example:

```
type MyInt int
```

Is `MyInt` also in the type set of `Integer`? Let's find out. Suppose we write a generic function that uses this constraint:

```
func Double[T Integer](v T) T {
    return v * 2
}
```

Can we pass it a `MyInt` value? We'll soon know:

```
fmt.Println(Double(MyInt(1)))
// MyInt does not implement Integer
```

No. That makes sense, because `Integer` is a list of named types, and we can see that `MyInt` isn't one of them.

How can we write an interface that allows not only a set of specific named types, but also any other types *derived* from them?

Type approximations

We need a new kind of type element: a *type approximation*. We write it using the tilde (~) character:

```
type ApproximatelyInt interface {
    ~int
}
```

The type set of `~int` includes `int` itself, but also any type whose underlying type is `int` (for example, `MyInt`).

If we rewrite `Double` to use this constraint, we can pass it a `MyInt`, which is good. Even better, it will accept *any* type, now or in the future, whose underlying type is `int`.

Derived types

Approximations are especially useful with struct type elements. Remember our `Pointish` interface?

```
type Pointish interface {
    struct{ x, y int }
}
```

Let's write a generic function with this constraint:

```
func Plot[T Pointish](p T) {
```

We can pass it values of type `struct{ x, y int }`, as you'd expect:

```
p := struct{ x, y int }{1, 2}
Plot(p)
```

But now comes a problem: we can't pass values of any *named* struct type, even if the struct definition itself matches the constraint perfectly:

```
type Point struct {
    x, y int
}
p := Point{1, 2}
Plot(p)
// Point does not implement Pointish (possibly missing ~ for
// struct{x int; y int} in constraint Pointish)
```

What's the problem here? Our constraint allows `struct{ x, y int }`, but `Point` is *not that type*. It's a type *derived* from it. And, just as with `MyInt`, a derived type is distinct from its underlying type.

You know now how to solve this problem: use a type approximation! And Go is telling us the same thing: "Hint, hint: I think you meant to write a ~ in your constraint."

If we add that approximation, the type set of our interface expands to encompass all types derived from the specified struct, including `Point`:

```
type Pointish interface {
    ~struct{ x, y int }
```

```
}
```

Exercise: A first approximation

Can you use what you've just learned to solve the `intish` challenge?

Here you're provided with a function `IsPositive`, which determines whether a given value is greater than zero:

```
func IsPositive[T Intish](v T) bool {
    return v > 0
}
```

(Listing `exercises/intish`)

And there's a set of accompanying tests that instantiate this function on some derived type `MyInt`:

```
type MyInt int

func TestIsPositive_IsTrueFor1(t *testing.T) {
    t.Parallel()
    input := MyInt(1)
    if !intish.IsPositive(input) {
        t.Errorf("IsPositive(1): want true, got false")
    }
}

func TestIsPositive_IsFalseForNegative1(t *testing.T) {
    t.Parallel()
    input := MyInt(-1)
    if intish.IsPositive(input) {
        t.Errorf("IsPositive(-1): want false, got true")
    }
}

func TestIsPositive_IsFalseForZero(t *testing.T) {
    t.Parallel()
    input := MyInt(0)
    if intish.IsPositive(input) {
```

```
        t.Errorf("IsPositive(0): want false, got true")
    }
}
```

(Listing `exercises/intish`)

GOAL: Your task here is to define the `Intish` interface.

HINT: A method set won't work here, because the `int` type *has* no methods! On the other hand, the type literal `int` won't work either, because `MyInt` is not `int`, it's a new type derived from it.

What kind of constraint could you use instead? I think you know where this is going, don't you? If not, have another look at the previous section on type approximations.

SOLUTION: It's not complicated, once you know that a type approximation is required:

```
type Intish interface {
    ~int
}
```

(Listing `solutions/intish`)

Interface literals

Up to now, we've always used type parameters with a *named* constraint, such as `Integer` (or even just `any`). And we know that those constraints are defined as interfaces. So could we use an *interface literal* as a type constraint?

Syntax of an interface literal

An interface literal, as you probably know, consists of the keyword `interface` followed by curly braces containing (optionally) some interface elements.

For example, the simplest interface literal is the empty interface, `interface{}`, which is common enough to have its own predeclared name, `any`.

We should be able to write this empty interface literal wherever `any` is allowed as a type constraint, then:

```
func Identity[T interface{}](v T) T {
```

And so we can. But we're not restricted to only *empty* interface literals. We could write an interface literal that contains a method element, for example:

```
func Stringify[T interface{ String() string }](s T) string {
    return s.String()
}
```

This is a little hard to read at first, perhaps. But we've already seen this exact function before, only in that case it had a *named* constraint `Stringer`. We've simply replaced that name with the corresponding interface literal:

```
interface{ String() string }
```

That is, the set of types that have a `String` method. We don't need to name this interface in order to use it as a constraint, and sometimes it's clearer to write it as a literal.

Omitting the `interface` keyword

And we're not limited to just method elements in interface literals used as constraints. We can use type elements too:

```
[T interface{ ~int }]
```

Conveniently, in this case we can omit the enclosing `interface { ... }`, and write simply `~int` as the constraint:

```
[T ~int]
```

For example, we could write some function `Increment` constrained to types derived from `int`:

```
func Increment[T ~int](v T) T {
    return v + 1
}
```

However, we can only omit the `interface` keyword when the constraint contains exactly one type element. Multiple elements wouldn't be allowed, so this doesn't work:

```
func Increment[T ~int; ~float64](v T) T {
// syntax error: unexpected semicolon in parameter list; possibly
// missing comma or ]
```

And we can't omit `interface` with method elements either:

```
func Increment[T String() string](v T) T {
// syntax error: unexpected ( in parameter list; possibly
// missing comma or ]
```

And we can only omit `interface` in a constraint *literal*. We can't omit it when defining a named constraint. So this doesn't work, for example:

```
type Intish ~int
// syntax error: unexpected ~ in type declaration
```

Referring to type parameters

We've seen that in certain cases, instead of having to define it separately, we can write a constraint directly as an interface literal. So you might be wondering: can we refer to T inside the interface literal itself? Yes, we can.

To see why we might need to do that, suppose we wanted to write a generic function `Contains[T]`, that takes a slice of T and tells you whether or not it contains a given value.

And suppose that we'll determine this, for any particular element of the slice, by calling some `Equal` method on the element. That means we must constrain the function to only types that have a suitable `Equal` method.

So the constraint for T is going to be an interface containing the method `Equal(T) bool`, let's say.

Can we do this? Let's try:

```
func Contains[T interface{ Equal(T) bool }](s []T,  v T) bool {
```

Yes, this is fine. In fact, using an interface literal is the *only* way to write this constraint. We couldn't have created some *named* interface type to do the same thing. Why not?

Let's see what happens if we try:

```
type Equaler interface {
    Equal(???) bool // we can't say 'T' here
}
```

Because the type parameter T is part of the `Equal` method signature, and we don't *have* T here. The only way to refer to T is in an interface literal inside a type constraint:

```
[T interface{ Equal(T) bool }]
```

At least, we can't write a *specific* interface that mentions T in its method set. What we'd need here, in fact, is a *generic* interface, and we'll see how to define that in the next chapter.

Exercise: Greater love

Your turn now to see if you can solve the `greater` exercise.

You've been given the following (incomplete) function:

```
func IsGreater[T /* Your constraint here! */](x, y T) bool {
    return x.Greater(y)
}
```

(Listing `exercises/greater`)

This takes two values of some arbitrary type, and compares them by calling the `Greater` method on the first value, passing it the second value.

The tests exercise this function by calling it with two values of a defined type `MyInt`, which has the required `Greater` method.

```
type MyInt int

func (m MyInt) Greater(v MyInt) bool {
    return m > v
}

func TestIsGreater_IsTrueFor2And1(t *testing.T) {
    t.Parallel()
    if !greater.IsGreater(MyInt(2), MyInt(1)) {
        t.Fatalf("IsGreater(2, 1): want true, got false")
    }
}

func TestIsGreater_IsFalseFor1And2(t *testing.T) {
    t.Parallel()
    if greater.IsGreater(MyInt(1), MyInt(2)) {
        t.Fatalf("IsGreater(1, 2): want false, got true")
    }
}
```

(Listing `exercises/greater`)

GOAL: To make these tests pass, you'll need to write an appropriate type constraint for `IsGreater`. Can you see what to do?

HINT: Remember, we got here by talking about constraints as interface literals, and in particular, interface literals that refer to the type parameter.

If you try to define some *named* interface with the method set containing `Greater`, for example, that won't work. We can't do it for the same reason that we couldn't define a named interface with the method set `Equal`: we don't know what type of argument that method takes.

Just like `Equal`, `Greater` takes arguments of some arbitrary type T, so we need an interface literal that can *refer* to T in its definition. Does that help?

SOLUTION: Here's one way to do it:

```
func IsGreater[T interface{ Greater(T) bool }](x, y T) bool {
    return x.Greater(y)
}
```

(Listing `solutions/greater`)

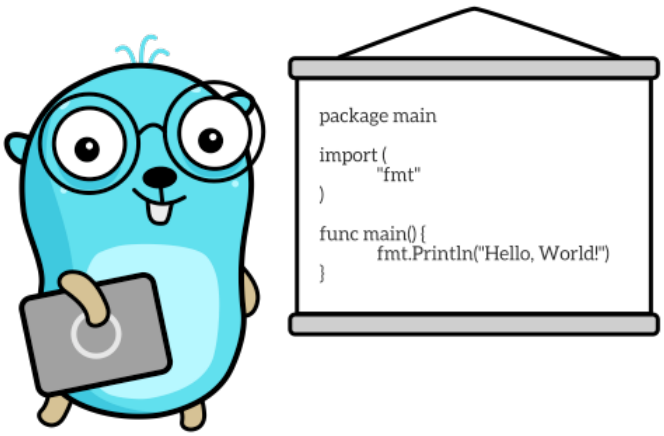
Like most things, it's delightfully simple once you know. For a type parameter T, the required interface is:

```
Greater(T) bool
```

And that's how we do that.

4. Operations

> *It is not only the programmer's responsibility to produce a correct program but also to demonstrate its correctness in a convincing manner.*
>
> —Edsger Dijkstra, "Notes on Structured Programming"



In the previous chapter we looked at different ways to constrain type parameters: method sets, named types, and approximations. And you'll recall that the reason we needed to constrain these types in the first place was so that we could do useful *operations* with values, like adding them together.

So what other operations might we want to do with values of parameterised types, and what constraints would make this possible? Let's take the addition example first.

Arithmetic

Remember the `Integer` interface we defined in the previous chapter? Let's extend it now to include all the built-in signed and unsigned integer types. While we're at it, let's use the ~ symbol to also extend this interface to include all types *derived* from those types, too:

```
type Integer interface {
    ~int | ~int8 | ~int16 | ~int32 | ~int64 | ~uint | ~uint8 |
        ~uint16 | ~uint32 | ~uint64
}
```

We should be able to use this constraint to write the generic `AddAnything` function we discussed in the first chapter, shouldn't we?

An `AddAnything` function

Here's one way to write the function, for example:

```
func AddAnything[T Integer](x, y T) T {
    return x + y
}
```

We're saying that `AddAnything`, for some integer type T, takes two T parameters, and returns a T result. Let's see if it works:

```
fmt.Println(AddAnything(1, 2))
// Output:
// 3
```

That's reassuring. Of course, it's possible to add together floating-point and complex numbers too, and right now our function can't do that. What we want is a more general (less constrained) type set that includes essentially all *numeric* types and their derivatives.

The set of all numeric types

Let's bring back our other interfaces from the previous chapter, `Float` and `Complex`, so that we can use all three type sets together:

```
type Float interface {
    ~float32 | ~float64
}

type Complex interface {
    ~complex64 | ~complex128
}
```

```
type Number interface {
    Integer | Float | Complex
}
```

Now we have a `Number` constraint that we can use to parameterise any function that operates on numbers. And let's update `AddAnything` to take this new constraint:

```
func AddAnything[T Number](x, y T) T {
    return x + y
}
```

This looks promising, and Go seems happy. Let's try adding two complex numbers, just for fun:

```
fmt.Println(AddAnything(1 + 2i, 3 + 4i))
// Output:
// (4+6i)
```

Exercise: Product placement

Here's another challenge for you: solve the `product` exercise.

Your job is to define a function `Product`, which will return the result of multiplying its two inputs. For example:

```
fmt.Println(product.Product(2, 3))
// Output:
// 6
```

That would be easy if the inputs were always `int`, for example, or any other specific numeric type, but I'm not here to make your life easy. Instead, the tests use `Product` to find the product of several *different* numeric types:

```
func TestProductOfInts2And3Is6(t *testing.T) {
    t.Parallel()
    want := 6
    got := product.Product(2, 3)
    if want != got {
        t.Errorf("want %d, got %d", want, got)
    }
}
```

```
func TestProductOfFloats1Point6And2Point3Is3Point68(t *testing.T) {
    t.Parallel()
    want := 3.68
    got := product.Product(1.6, 2.3)
    if math.Abs(want-got) > 0.0001 {
        t.Errorf("want %f, got %f", want, got)
    }
}

func TestProductOfComplex2Plus3iAnd3Plus2iIs0Plus13i(t *testing.T) {
    t.Parallel()
    want := 0 + 13i
    got := product.Product(2+3i, 3+2i)
    if want != got {
        t.Errorf("want %v, got %v", want, got)
    }
}
```

(Listing `exercises/product`)

So in order to write `Product`, you'll also need to define a suitable type constraint for this function. Make sure it's broad enough to allow all the different numeric types supplied by the tests, but not *so* broad that it doesn't allow you to multiply the arguments together.

GOAL: Get the tests passing!

HINT: Actually, we've done most of the necessary work already, haven't we? Any type that satisfies `Number` is multipliable, so that seems a reasonable constraint.

SOLUTION: Straightforwardly, then, we can write:

```
type Integer interface {
    ~int | ~int8 | ~int16 | ~int32 | ~int64 | ~uint | ~uint8 |
        ~uint16 | ~uint32 | ~uint64
}

type Float interface {
```

```
    ~float32 | ~float64
}

type Complex interface {
    ~complex64 | ~complex128
}

type Number interface {
    Integer | Float | Complex
}

func Product[T Number](x, y T) T {
    return x * y
}
```

(Listing `solutions/product`)

Ordered types

So much for addition. What else could we do? How about finding the greater of two numbers, for example?

```
func Greater[T Number](x, y T) T {
    if x > y {
        return x
    }
    return y
}
```

This *looks* reasonable, but doesn't compile:

```
invalid operation: x > y (type parameter T is not comparable
with >)
```

The > operator

What's going on? We know that at least some of the types in `Number`'s type set support the > operator. For example, integers certainly do, and so do floats.

But if we're going to use the > operator with values of T, then *all* possible types for T must support that operator, and clearly not all of them do. By elimination, it must be

`Complex` that's the problem.

Unlike the *real* (non-complex) numbers, there's no natural ordering of complex numbers. Therefore, they can't be compared with the > operator to see which is greater.

So while the `Number` constraint works fine for addition, we'll need a more restrictive constraint for `Greater`. We'll need one that includes the *real* numbers (integer and floating-point), but not the complex numbers.

Well, we can write that explicitly, as the union of these two type sets we've already defined:

```
type Real interface {
    Integer | Float
}
```

This looks promising, so let's update our `Greater` function to use it:

```
func Greater[T Real](x, y T) T {
```

Let's try it with the integers 1 and 2, to find out which one is bigger. Look away now if you don't want to see the result:

```
fmt.Println(Greater(1, 2))
// Output:
// 2
```

Strings

The > operator, and its friends, isn't limited to just numbers; we can also use it with strings. For example, the string `"b"` is greater than the string `"a"`, from Go's point of view.

So we can make our `Greater` function more widely useful by accepting not only real numbers, but also strings and types derived from `string`. In other words, all types that support the > operator.

Let's call these, in general, *ordered* types. For a type to be ordered, there must exist some well-defined way to arrange values of that type in order, from smallest to largest.

That's really what it means to use the > operator with two values, isn't it: to ask which one comes first in this well-defined ordering. How you decide that is different for each ordered type, but the point is that you *can* always decide.

An `Ordered` interface

Here's one way we can write an interface whose type set is the ordered types:

```
type Ordered interface {
    Integer | Float | ~string
}
```

Recall that the ~ specifies a type approximation: we're allowing not just `string`, but any type whose underlying type is `string`, too.

Let's update our `Greater` function to use the `Ordered` constraint, then:

```
func Greater[T Ordered](x, y T) T {
    if x > y {
        return x
    }
    return y
}
```

This compiles okay, so we'll try it out with a couple of strings:

```
fmt.Println(Greater("a", "b"))
// Output:
// b
```

Actually, there's a better way to write this, using the built-in `max` function:

```
func Greater[T Ordered](x, y T) T {
    return max(x, y)
}
```

We wouldn't bother to write this function in practice, of course, since Go already provides it! But it's clear that `max` (and its friend `min`) must do the same thing under the hood as the > (and <) operators do. Thus, the `Ordered` constraint is required here too.

The `cmp.Ordered` constraint

It's becoming clearer why we might want to constrain type parameters in certain ways, isn't it?

We need to restrict the allowed types enough that we can guarantee that they'll support the operator we're using (such as + or >).

On the other hand, we don't want to restrict them *too* much. To make our function as widely usable as possible, we should include *all* the types that support the relevant operator.

Our `Ordered` constraint, then, expresses precisely the set of types that can be used with `>`, its related operators `<`, `>=`, and `<=`, and the built-in functions `max` and `min`. Very useful!

So useful, indeed, that it's defined for us in the standard library. If we import the `cmp` package, we can refer to this constraint as `cmp.Ordered`:

```
import "cmp"

func Greater[T cmp.Ordered](x, y T) T
```

The nice thing about this is, not only do we not have to define this constraint ourselves, the standard library version is guaranteed to be up-to-date. Even if Go introduces new built-in ordered types in the future, they'll satisfy `cmp.Ordered`.

Multiple type parameters

So far we've seen generic functions with a single type parameter, and you might be wondering if it's possible to write functions with *multiple* type parameters. Indeed it is, and it works just the way you'd expect.

Function on two (or more) types

Let's write a version of our `Identity` function that takes values of *two* arbitrary types, instead of just one, and returns them both:

```
func Identity[T, U any](x T, y U) (T, U) {
    return x, y
}
```

This hurts the eyes a little at first, until you get used to reading generic code. Let's see what it looks like in use:

```
fmt.Println(Identity(1, "hello"))
// Output:
// 1 hello
```

This is fine, but we're rather limited in what we can do with T and U values *together*, because we don't have any idea of the *relationship* between the two types T and U.

Each type parameter is a distinct type

For example, we can't find which value is greater, using the `>` operator. Why not? Because T and U are different types, and that operator can only work on values of the

same type. For the same reason, we couldn't add them together, or use any other operators.

In functions with multiple type parameters, then, each of those abstract types is distinct to Go. Even if they have the same *constraint*, that doesn't guarantee they'll be the same *type*.

That might happen to be the case, by coincidence, but Go has no way to guarantee this. It's much more likely that T and U will be two different types, each of which satisfies the constraint in a different way.

For example, `int` and `float64` both satisfy `cmp.Ordered`, but they're not the same type, and we couldn't compare them with `>`. So if we want to use an operator like that, we'd better take two values of a *single* arbitrary type T, rather than two type parameters T and U.

Functions on slice types

Here's another interesting case where multiple type parameters can be helpful. Consider a generic function that takes a slice of some arbitrary element type, such as the `Len` function we saw in an earlier chapter:

```
func Len[E any](s []E) int {
    return len(s)
}
```

And suppose we've defined some composite type—`StringList`, let's say—which is just a slice of strings:

```
type StringList []string
```

Can we pass a value of `StringList` to `Len`? Certainly:

```
fmt.Println(Len(StringList{"a", "b", "c"}))
// Output:
// 3
```

So far, so good. But what if we want to write some function that takes a slice and *returns* a slice of the same type? For example:

```
func Identity[E any](s []E) []E {
    return s
}
```

This works too, or so it seems:

```
fmt.Println(Identity(StringList{"a", "b", "c"}))
// Output:
// [a b c]
```

The problem with derived slice types

But there's a hidden problem here. Let's use the `%T` verb with `fmt.Printf` to print out the *type* of the result returned by `Identity`:

```
result := Identity(StringList{"a", "b", "c"})
fmt.Printf("result is a %T\n", result)
// Output:
// result is a []string
```

Shouldn't that be `StringList`? At least, that's what we might have expected. We *passed* a value of type `StringList`, and the function returned the exact slice we passed to it. But now it's changed type!

We can see why this is happening if we replace the generic type parameter with the specific type involved. In this case, it's `string`:

```
func Identity(s []string) []string
```

Now, it's perfectly okay to pass a `StringList` value to this function, since `StringList` is defined as `[]string`. That's just how Go types work. But the declared *result* type is `[]string`, and that's what we'll get as the result: that's how Go *functions* work!

Does this matter? Well, yes. `StringList` is *not* the same type as `[]string`, even though they're mutually convertible.

For example, if we defined some method on `StringList`, we wouldn't be able to call it on `result`, because that's a different type:

```
func (s StringList) Len() int {
    return len(s)
}

func main() {
    result := Identity(StringList{"a", "b", "c"})
    fmt.Println(result.Len())
}
// result.Len undefined (type []string has no field or method Len)
```

Oh dear! What shall we do?

Parameterizing by slice and element type

Well, we really want any function of this kind to return the exact same type as it receives, even if that's a derived type such as `StringList`. One way to write that is to add another type parameter, S, which is itself defined in terms of E:

```
func Identity[S []E, E any](s S) S
```

We're saying that `Identity` takes a parameter of type S, where S is a slice of any type E. Fair enough. There's no important difference between this and what we had before. The point is that we can now declare the function's result type as S, whatever S turns out to be.

Now does it work the way we want?

```
result := Identity(StringList{"a", "b", "c"})
fmt.Println(result.Len())
// StringList does not satisfy []string (possibly missing ~ for
// []string in []string)
```

Almost! Go is reminding us that, since `StringList` is derived from `[]string`, we need to add a type approximation, using the ~ symbol:

```
func Identity[S ~[]E, E any](s S) S {
    return s
}
```

Now it works:

```
result := Identity(StringList{"a", "b", "c"})
fmt.Println(result.Len())
// Output:
// 3
```

And I need to know this because...?

Don't worry if this doesn't quite land with you first time: it *is* confusing. Sorry about that.

Suffice it to say that, when we're writing functions that take slice types and return either a slice or an element of the same type, we need to be a little bit careful. While we might

be able to get *away* with a plain old `[E any]` constraint most of the time, it makes our code less flexible than it should be.

Instead, we should use this more sophisticated `[S ~[]E, E any]` style to make sure that our function will behave correctly even in the case of derived slice types. The `StringList` example makes it clear what can go wrong if we don't do this.

Later in this book we'll introduce the new standard library `slices` package, and when you look at the documentation you'll notice that any functions that return slices are defined in just this way:

```
func Clone[S ~[]E, E any](s S) S
```

Now you know why, and you can use the same idea in your own functions.

Comparable types

Probably the most common operation we do with two values in Go programs is *comparing* them: that is, checking whether they're equal. How could we do that in a generic function?

Not every type is comparable

We might start by trying to use the `any` constraint. Let's try:

```
func Equal[T any](x, y T) bool {
    return x == y
}
// invalid operation: x == y (incomparable types in type set)
```

Oh dear. It's the same kind of problem we've run into before, isn't it? One does not simply compare two values of any type with the == operator, because that operator is not defined on all types. For example, it's not defined on slices, maps, channels, or functions.

So `any` is too broad a constraint to allow us to use the == operator. We need to specify a smaller set of types.

Not every comparable type is ordered

Well, what about `Ordered`, then? Surely every type that's `Ordered` can be used with ==?

```
func Equal[T cmp.Ordered](x, y T) bool {
    return x == y
```

```
}
```

Indeed, and this works:

```
fmt.Println(Equal(1, 2))
// Output:
// false
```

But wait a minute: have we been *too* restrictive? The point of generic functions is to accept as wide a range of types as possible. The type set of `Ordered` might not include everything we can compare with the == operator.

So are there any types that we're missing out with this constraint? That is, types that don't support > but *do* support ==?

There are infinitely many comparable types

Actually, yes, there are many such types. `bool` is one example. We can compare two `bool` values for equality with ==, but we can't order them with >. (Which is bigger, `true` or `false`? That question doesn't even make sense.)

Structs are another example of *comparable* types that nevertheless aren't ordered. And there are many more such "comparable, but unordered" types: complex numbers, pointers, channels, arrays, and interfaces.

So for a function like `Equal` that uses the == operator, we need a constraint that specifies the full set of comparable types. How could we write that as an interface?

Perhaps surprisingly, it turns out that we can't!

Yes, we could list all the built-in comparable types by name, but what about user-defined structs, interfaces, channels, and so on?

We can't use a method set, because the == operator doesn't *call* methods; no operator does. We can't use type elements either, because we can't name every possible struct type, for example.

It turns out that the set of comparable types is just infinitely large. Even if you knew all their names, which you don't, you'd get very tired by the time you'd written them all out as an explicit type set.

Fortunately, you don't have to do that, as we'll see.

The `comparable` constraint

It's clearly very important to be able to compare values using ==. We do this all the time in Go programs, so not being able to do it in generic functions would be a severe limitation, to say the least.

This is why Go provides a predeclared constraint named `comparable`. It specifies exactly the set of comparable types, which as we've seen we wouldn't be able to define using a Go interface.

It sounds like `comparable` is exactly the type constraint we need for our `Equal` function:

```
func Equal[T comparable](x, y T) bool {
    return x == y
}
```

No errors, so let's try it out:

```
fmt.Println(Equal(1, 1))
// Output:
// true
```

What a relief!

Why is `comparable` predeclared?

You might be wondering why `comparable` is a predeclared identifier in Go, rather than being provided in some standard library package, like `cmp`. Why isn't it `cmp.Comparable`, for example?

Well, because that package is written in pure Go, and we already know that we can't *write* `comparable` in Go. It has to be implemented internally by the compiler, which can already type-check expressions using the == operator to make sure they're allowed. This works the same way.

Exercise: Duplicate keys

See if you can use this idea to tackle the dupes exercise.

The task here is to determine if a given slice contains duplicated elements. For example, if a slice of numbers contains any given number more than once, then the slice contains duplicates. The duplicate elements need not be consecutive.

To do this, you'll need to write a function `Dupes`, with an appropriate type constraint, that returns `true` if the given slice contains duplicates, or `false` otherwise.

For example:

```
fmt.Println(dupes.Dupes([]int{1, 2, 3}]
// Output:
// false
fmt.Println(dupes.Dupes([]int{1, 2, 1}]
```

```
// Output:
// true
```

Of course, to make things a bit more fun (for certain values of "fun") the tests will call `Dupes` with some different, but still comparable, types:

```
func TestDupesIsTrueGivenNonConsecutiveDuplicates(t *testing.T) {
    t.Parallel()
    s := []int{1, 2, 3, 1, 5}
    if !dupes.Dupes(s) {
        t.Errorf("Dupes(%v): want true, got false", s)
    }
}

func TestDupesIsFalseGivenNoDuplicates(t *testing.T) {
    t.Parallel()
    s := []string{"a", "b", "c"}
    if dupes.Dupes(s) {
        t.Errorf("Dupes(%v): want false, got true", s)
    }
}
```

(Listing `exercises/dupes`)

GOAL: Get the tests passing!

HINT: We've been talking about comparable types, and how to use the predeclared identifier `comparable` to constrain functions that need to check the equality of their arguments with ==.

Well, this sounds just like one of those functions, doesn't it? Specifically, the `Dupes` function needs to look at every slice element in turn, and if that same value has occurred before, then the slice contains dupes.

In principle, that means comparing each slice element with every other, but maybe it's not really that hard. It would help if we had a way of storing each unique value that we've seen so far, and quickly checking if a given value is in that set. Can you think of a Go data structure that would make that easy?

SOLUTION: Well, here's one possibility:

```
func Dupes[E comparable](s []E) bool {
    seen := map[E]bool{}
    for _, v := range s {
        if seen[v] {
            return true
        }
        seen[v] = true
    }
    return false
}
```

(Listing `solutions/dupes`)

Note that we don't need to worry about derived slice types here, as we did with `Identity`, for example. That's because `Dupes` always returns `bool`, not some element of the slice, or some new slice of the same type.

Abstract types

Armed with `comparable` and the `cmp.Ordered` constraint, then, we feel it should now be possible to write some interesting non-trivial generic functions. Let's try.

A `Greatest` function

We already have a function to find the greater of two values, so what about one to find the greatest element of a *slice* of values?

Let's call this function `Greatest`, then. How should we constrain the element type of the slice it operates on? To answer that question, we need to ask, first, what are we going to *do* with those values?

Finding the biggest element of a slice is a lot like finding the bigger of two values; in fact, that's exactly how we're going to do it.

We'll start by checking if the slice is empty, because in that case it makes no sense to ask for its biggest element. Otherwise, we'll compare each element in turn with the biggest value we've seen so far, and to do that we'll use the > operator.

This sounds like a job for the `cmp.Ordered` constraint, doesn't it? Here goes:

```
func Greatest[S ~[]E, E cmp.Ordered](s S) E {
    if len(s) < 1 {
        panic("Greatest: empty slice")
    }
```

```
    var top E
    for _, e := range s {
        top = max(top, e)
    }
    return top
}
```

As with the `Greater` function we wrote earlier, because we know that `E` is ordered, we can use `max` to find which of any two values is the greater.

Spoiler alert, but we don't actually need to write this function ourselves, even though now we know how. It's included in the `slices` package, which we'll talk more about later:

```
s := []string{"faith", "hope", "love"}
fmt.Println(slices.Max(s))
// Output:
// love
```

(Another of life's great questions answered!)

The scope of type parameters

Here's something interesting we haven't seen before. We didn't just use the type parameter `E` in the *signature* of the `Greatest` function. We used it in the body too, to declare our result variable:

```
var top E
```

This is the first time we've used a type parameter within the body of a function like this, but it's absolutely fine. We already know we can refer to `E` in the function's parameter and result lists, and this is no different.

In fact, the *scope* of a type parameter extends as far as the end of the function body. When it's used in the body like this, we call `E` an *abstract type*.

There's nothing special about an abstract type. When the function is instantiated, it'll just turn out to be some ordinary specific type: in the example we ran just now, it was `string`. So this declaration becomes equivalent to just:

```
var top string
```

The zero value

Sometimes in a Go function we want to return the *zero value* of whatever type we're dealing with (in the error case, for example). How can we do that with an abstract type like E?

Well, we already know that all types in Go have a *default* value, which is the value a variable has if we haven't yet assigned anything to it.

That's just another name for the zero value, so using a `var` statement is one way to get a zero E:

```
var top E
return top
```

This isn't always convenient, so another possible way is to explicitly *convert* the constant 0 to E:

```
return E(0)
```

Provided that 0 is *representable* in E, this is okay. In other words, if we could convert the constant 0 to every possible type E allowed by the constraint, then we can do this.

That's not always the case, but when it is, `E(0)` is a convenient way to get the zero value. When it's not, you can use `var` instead.

Switching on abstract types

We saw in the first chapter that when we have some ordinary parameter `x` of an interface type (`any`, let's say), we can use a type switch to find out exactly what its concrete type is, and take the appropriate action:

```
switch v := x.(type) {
case int:
    return v + y
case float64:
    return v + y
case ...
```

So you might be wondering if we can pull the same switch, so to speak, with a parameter of some abstract type. Sorry, but that pig won't hunt:

```
func Identify[T any](x T) {
    switch x.(type) {
    case int:
```

```
        fmt.Println("Looks like an int")
    case float64:
        fmt.Println("Looks like a float")
    default:
        fmt.Println("Doesn't look like anything to me")
    }
}
// cannot use type switch on type parameter value x (variable of
// type T constrained by any)
```

Although the error message doesn't really make it clear, the problem is not that T is an abstract type. It's that type switches only work on *interface* types, and Go can't guarantee that T will be an interface type.

It *might* be, sure, but a lot of things *might* be that ain't so. Since T can be `any`, it could certainly be some concrete type such as `int`, and we can't type-switch on concrete types: that wouldn't mean anything. The point of a type switch is precisely to ask "Given this value of some interface type, what is its concrete type?"

And, as we discussed, there's usually no reason to write a type switch in a generic function. Instead, we'd write a different version of the function for each specific type that we know how to handle: `int`, `float64`, and so on. Alternatively, we could write a plain old function that takes an interface type, such as `any`. Either way, no generics needed.

But, if the need should arise, there *is* a way to do this, and it's very simple. Just convert the value to `any`, then switch on it!

```
func Identify[T any](x T) {
    switch any(x).(type) {
    case int:
        fmt.Println("Looks like an int")
    case float64:
        fmt.Println("Looks like a float")
    default:
        fmt.Println("Doesn't look like anything to me")
    }
}

func main() {
    Identify(99)
}
// Output:
```

```
// Looks like an int
```

5. Types

> *Typing is no substitute for thinking.*
> —"A Manual for BASIC"

So far in this book we've talked mostly about generic functions. Let's talk a little now about generic *types*.

Named types

Go's type system can help us write correct programs, by catching situations where we accidentally use a value of the wrong type. For example, maybe there's a function that takes an `int` parameter, but we tried to pass it a `float64`. Go is always at your elbow to point out that kind of thing, in a friendly way, and it's very helpful.

There are often lots of variables in a given program, and it's easy to get them mixed up, especially if their names are unhelpfully brief. We might mistakenly use the variable `a` as someone's age, for example, when it actually holds their address. Go can spot this kind of problem for us, because these values would be of different types.

And we can extend this idea, creating new types to represent different kinds of information, even when they have the same *underlying* type. Let's see an example.

Named basic types

Suppose we want to represent data about a person, including their age and height. We could use `int` for both of these fields, but then there's a chance we could get the two integers mixed up. Go wouldn't catch that problem, because it's not a type mismatch: it's a *meaning* mismatch.

If our program identifies someone as 30 centimetres tall and 180 years old, for example, it's probably the result of such a mistake (though you never know). To prevent this, we can declare new named types to represent the two different kinds of information:

```
type (
    Age      int
    HeightCM int
)
```

Now Go can help us catch errors of the "senior citizen of restricted growth" kind, by type-checking all references to age and height. If we try to pass a value of type `Age` where `HeightCM` is expected, the program won't compile.

Generic basic types

Can we use this trick to give new names to generic types, then? For example:

```
type MyT[T any] T
// cannot use a type parameter as RHS in type declaration
```

No, this isn't allowed. While there are good reasons to give new names to basic types such as `int`, as we've just seen, this is different. We don't *know* what the basic type T is yet, so defining a new type based on it wouldn't be useful.

Generic slice types

On the other hand, defining new *composite* types, such as slices, is entirely possible, as we saw in the previous chapter:

```
type Bunch[E any] []E
```

We've also seen that we need to *instantiate* such types when we use them:

```
b := Bunch[int]{1, 2, 3}
```

And generic types, as you'll also recall, are always instantiated. That is, in your compiled program, an abstract type such as E becomes *some specific type*, such as `int`.

That means that a generic slice type like `Bunch`, for example, *cannot contain elements of different types*. That's because there's no such *type* as just `Bunch`. It has to be instantiated, becoming, for example, a `Bunch[int]`:

```
b := Bunch[int]{1, 2, 3}
b = append(b, "hello")
// cannot use "hello" (untyped string constant) as int value
// in argument to append
```

It's no surprise that we can't append a string to a `Bunch[int]`, any more than we can do the same to a `[]int`. So, even though we refer to the `Bunch` type as "generic", it can still only contain elements of one *specific* type for any given `Bunch` value, not a mish-mash of values of different types.

There are no generic types

Remember how we talked about a confusing overlap between basic interfaces and type constraints? This especially applies to generic types, because we *can* perfectly well write some definition like:

```
type Stuff []any
```

This really *is* a generic slice, if you like: we can put values of any type into it, including values of *different* concrete types. For example:

```
s := Stuff{1, 2, 3}
s = append(s, "hello")
fmt.Println(s)
// Output:
// [1 2 3 hello]
```

Exactly what we can't do with a so-called "generic" slice type like `Bunch`! So it's critical to be really clear in the way we think and talk about generic types, especially collection types.

Generic types are *not* like collections of empty interface values. Once they're instantiated, they are just perfectly ordinary collections of elements, all of the same specific type.

We can talk about "generic types", then, when we're speaking loosely, but to be absolutely precise, *there are no generic types* in Go—at least, not in a running Go program.

Just as with functions, at compile time all parameterised types must be instantiated on some specific type. If Go can't work out, or we haven't said explicitly, what type that is,

then the program can't be compiled. At *run* time, therefore, there can only be specific types.

Slices of interface types

Just to add to the potential confusion, generic types *can* be instantiated on interface types, provided always that the interface in question matches the necessary constraint.

For example, we've instantiated `Bunch` on concrete types like `string` or `int`. But we're not restricted to only concrete types. We could perfectly well instantiate `Bunch` on some interface type, such as `error`:

```
var b Bunch[error]
b = append(b, errors.New("oh no"))
fmt.Println(b)
// Output:
// [oh no]
```

It's important to understand why the following two things are fundamentally different:

1. A slice of elements of type `any`
2. A slice of elements of type E, where E is `any`.

This is a good example of where clarity of terminology matters, because people often talk loosely about "generic container types", for example, meaning something like a slice of `any`, which can contain elements of different concrete types.

That's not what a `Bunch` is, as we've seen. Rather, a `Bunch` is a slice of elements that all have the *same,* arbitrary type.

Which type that *is* will be decided when we actually use the type somewhere in our program: for example, it might be `int`, if we use a `Bunch[int]`.

Generic map types

If parameterised slice types are okay, what about maps? It feels like we should be able to create map types where either the key type or the element type (or both) are arbitrary, too. Can we?

Maps of abstract element types

For example, can we declare a type based on a map of `string` to some arbitrary element type V? Certainly we can:

```
type Catalog[V any] map[string]V
```

Let's try it out:

```
cat := Catalog[int]{}
fmt.Println(cat["bogus"])
// Output:
// 0
```

We've instantiated `Catalog[V]` with the type `int`, creating a map of `string` to `int`.

The result of this is exactly the same as if we'd written something like:

```
type CatalogInt map[string]int
cat := CatalogInt{}
fmt.Println(cat["bogus"])
```

Multiple type parameters

We saw in the previous chapter that it's possible to define functions with multiple type parameters, and we can do the same thing when defining types. For example, since maps have a key type *and* an element type, could we use type parameters for both?

Let's try:

```
type Index[K, V any] map[K]V
// invalid map key type K (missing comparable constraint)
```

Hmm, what's gone wrong here? It doesn't like us constraining K by `any`. Actually, that makes sense. We can't define an ordinary, non-generic Go map with the `any` interface type, and even some specific types are disallowed.

Why? Think about how maps work. If we store a value in the map with a certain key, we expect to be able to retrieve the same key again later. But how can we tell that it *is* the same key unless we can use the == operator with that type?

We can't, so that restricts map keys to only those types that support the == operator. As we've seen, that constraint is described by the predeclared identifier `comparable`, and that's why Go complains:

```
missing comparable constraint
```

Whatever constraint we impose on K, then, it must be at least `comparable`. If we add that constraint (for K only), then it should work:

```
type Index[K comparable, V any] map[K]V
// this is fine
```

Instantiating multiple type parameters

How do we explicitly *instantiate* an `Index`, then? We already know how to instantiate generic types with one type parameter. In that case, we put the type name in square brackets, like `Bunch[int]`.

And it's just the same when we have two type parameters. We put them both inside the square brackets, separated by a comma:

```
age := Index[string, int]{}
```

Generic struct types

It seems like type parameters could be useful in defining generic struct types, too. Could we, for example, define some struct type with a field of arbitrary type T?

```
type NamedThing[T any] struct {
    Name  string
    Thing T
}
```

This seems straightforward, and we can instantiate and use it without any difficulty:

```
n := NamedThing[float64]{
    Name:  "Latitude",
    Thing: 50.406,
}
fmt.Println(n)
// Output:
// {Latitude 50.406}
```

Self-reference

What if we tried to do something silly here? For example, instantiate a `NamedThing` on the `NamedThing` type itself? Would we disappear into a never-ending recursive hall of mirrors?

That sounds fun, so let's try:

```
var n NamedThing[NamedThing]
// *Inception horn*
```

No, because, as we saw earlier, there's really *no such type as* `NamedThing`. There are only specific *instantiations* of it: for example, `NamedThing[float64]`.

To instantiate the outer `NameThing`, we need to give a specific type in square brackets, so we can't just say `NamedThing`. That's a *generic* type, which is no good:

```
cannot use generic type NamedThing[T any] without instantiation
```

So we have to instantiate the inner `NamedThing`, and that eliminates the recursion:

```
p := NamedThing[NamedThing[float64]]{
    Name: "Position",
    Thing: NamedThing[float64]{
        Name:  "Latitude",
        Thing: 50.406,
    },
}
fmt.Println(p)
// Output:
// {Position {Latitude 50.406}}
```

Maybe this particular example is a little silly, or maybe not, but it's a good way of making the point, and understanding the rules of generic types. It's quite possible that real applications might want to nest generic types in this way.

Methods

Now that we know how to define generic types, including slices, maps, and structs, what about methods on those types? Are they allowed?

Adding methods to generic types

Take the `Bunch` type, for example, based on a slice. Could we write a `First` method that returns the first element of the bunch? Let's try:

```
type Bunch[E any] []E

func (b Bunch[E]) First() E {
    return b[0]
}
```

It's not hard to *write* the `First` method, but the win for generics is that now we won't have to write it N times for N types. We can write it once, and use it on any kind of

`Bunch` that we choose to instantiate:

```
b := Bunch[string]{"a", "b", "c"}
fmt.Println(b.First())
// Output:
// a
```

Exercise: Empty promises

Have a look at the empty exercise and see if you can solve it, based on what you just learned.

Your job is to create a `Sequence` type that can hold multiple values, and an `Empty` method on it that will report whether or not the sequence is empty. It will need to pass these tests, instantiated on two different types:

```
func TestEmptyIsTrueForEmptySequence(t *testing.T) {
    t.Parallel()
    s := empty.Sequence[int]{}
    if !s.Empty() {
        t.Fatalf("Empty(%v): want true, got false", s)
    }
}

func TestEmptyIsFalseForNonEmptySequence(t *testing.T) {
    t.Parallel()
    s := empty.Sequence[string]{"a", "b", "c"}
    if s.Empty() {
        t.Fatalf("Empty(%v): want false, got true", s)
    }
}
```

(Listing `exercises/empty`)

GOAL: Get the tests passing!

HINT: Actually, we've done most of the necessary work already, haven't we? You can create a generic `Sequence` type, parameterised by some suitable constraint, and write an `Empty` method on it. There's nothing new required here: it's just consolidating what you already know.

SOLUTION: Here's my version:

```
type Sequence[E any] []E

func (s Sequence[E]) Empty() bool {
    return len(s) == 0
}
```

(Listing `solutions/empty`)

Parameterised methods

If we can write methods on generic types, and we already know we can write generic *functions*, then could we write a generic *method* on a generic type?

That is, could we write a method on a type such as `Bunch[E]` that takes a parameter of some *other* arbitrary type? Let's try an example.

Suppose we wanted to create a method `PrintWith` that prints the contents of the `Bunch[E]` along with some other value of an arbitrary type T. How would we write that?

It turns out that we can't:

```
func (b Bunch[E]) PrintWith[T any](v T) {
    fmt.Println(v, b)
}
// methods cannot have type parameters
```

Well, there's our answer! Methods cannot have type parameters in Go. Sorry about that.

More generic composite types

We've looked at generic slices, maps, and structs, but those aren't the only kind of types available in Go. What about interfaces, for example?

Interfaces

Remember our `Equaler` example from the previous chapter, when we had to write our constraint as an interface literal, because it referred to T?

```
func Contains[T interface{ Equal(T) bool }](s []T,v T) bool {
```

We couldn't define a named interface `Equaler`, because it would have to refer to T in its method set, and we don't *have* T at that point!

But it seems completely logical that we should be able to define some *generic* interface type `Equaler[T]`, doesn't it?

In other words, we should be able to write this:

```
type Equaler[T any] interface {
    Equal(T) bool
}
```

Absolutely! For any type T, `Equaler[T]` is an interface specifying a method `Equal` that takes a T parameter and returns `bool`. And that's just what we wanted.

Channels

Every Go programmer loves channels, and it would be disappointing if we couldn't create a channel of some (instantiated) generic type.

Let's try:

```
type myChan[E any] chan E
ch := make(myChan[error])
go func() {
    ch <- errors.New("oh no")
}()
fmt.Println(<-ch)
// Output:
// oh no
```

Of course we can! And since there are many operations we can do on channels without caring about the element type, this is very useful. For example, the `Drain` and `Merge` examples we saw in an earlier chapter use generic channels.

6. Functions

> *Most of the arguments for adding generics to Go are to support collection types, sets, tries, etc. That isn't very interesting to me, I'm sure it's interesting to others, but not to me; maps and slices work just fine. What is interesting to me is writing generic functions.*
>
> —Dave Cheney

Now that we've trotted patiently through several chapters explaining what generic functions and types even *mean*, and how they work in Go, let's break into a gentle canter through some *applications* of generics.

After all this fuss, what real-world code can we actually write with type parameters? Are they of any practical use in programs?

Functions on container types

What did the people who so loudly and persistently campaigned for generics in Go *want*, actually? What was it they needed to write that they couldn't do (easily) without generics?

Well, as we already discussed, it was difficult to write general-purpose functions on Go's built-in *collection*, or *container* types, slices and maps, without generics. Why? Well, be-

cause there's no such type as `slice`, for example: every slice type is a slice *of* some element type, such as `int`.

Any non-generic function that takes a slice, then, must take a slice of some *specific* type, which means that we have to duplicate it for every distinct type that we want to handle.

Worse, we can't define some new type (such as `MyInt`) and pass a slice of it to one of these functions. We'd have to implement the function all over again for every such type.

Contains

One important consequence of generics, then, is the ability to write *utility packages* providing functions on arbitrary container types, such as slices. For example, without generics it wasn't possible to provide a packaged set of common operations on a slice, such as `Contains`: a function that checks if the slice contains a certain element.

Instead, everybody who wanted to *do* that operation in their programs just had to write it anew each time, for the specific slice type they were using.

It's not hard to *implement* `Contains`, as we'll see. It's just that, without generics, it's hard to write a single version of `Contains` that works with multiple slice types.

Happily, that gap is now filled, and we can write a generic `Contains`! Let's try.

```
func Contains[E comparable](s []E, v E) bool {
    for _, vs := range s {
        if v == vs {
            return true
        }
    }
    return false
}
```

For any comparable type E, `Contains[E]` takes a slice of E and an E value, and returns `bool`.

We know that the element type must be constrained by at least `comparable`, because we'll be comparing each element against `v` to see if it's the one we're looking for.

Reverse

What about other operations on generic slices? We've already written a `Greatest` function in a previous chapter, and we feel `Least` should also be possible along the same lines.

Other common operations on slices are `Sort` and `Reverse`. Let's tackle `Reverse` first, because it's easier.

```
func Reverse[S ~[]E, E any](s S) S {
    result := make(S, 0, len(s))
    for i := len(s) - 1; i >= 0; i-- {
        result = append(result, s[i])
    }
    return result
}
```

This time, we don't need `comparable`, because we won't be doing any comparisons. All we need to do is loop over the input slice backwards, appending each element to the result slice.

The specific implementation doesn't matter, and this one may not be the most efficient or even the easiest to understand. The point is that this is something we simply couldn't have written at all without generics, without repeating the whole function for every specific element type.

`Sort`

Sorting a slice is another example of something which can be a bit laborious without generics.

The standard library's `sort.Slice` API requires users to pass in their own function to determine which of two elements is the lesser:

```
sort.Slice(s, func(i, j int) bool {
    return s[i] < s[j]
})
```

But it seems silly to have to pass in such a function when we're sorting a slice of something perfectly straightforward like `int`, and that's usually the case. Go knows perfectly well how to compare two integers with <, so why should we have to write a function to do that?

Well, you know why, but that doesn't make it any less annoying. Now, though, we can write a generic `Sort` function which doesn't have such an awkward API:

```
func Sort[S ~[]E, E cmp.Ordered](s S) S {
    result := make(S, len(s))
    copy(result, s)
    sort.Slice(result, func(i, j int) bool {
        return result[i] < result[j]
    })
```

```
    return result
}
```

Again, this certainly isn't a model implementation—it uses `sort.Slice` under the hood, which is slow because it uses reflection. That's not necessary with generics, and we could implement some efficient sort algorithm such as Quicksort directly.

The important thing about this `Sort` function is not whether the code is any good—it isn't—but that we can write it straightforwardly for any `Ordered` type. So that unordered types needn't feel left out, we could always provide a version where the user can pass in their own `Less` function, as before.

First-class functions

As we discussed in the earlier chapter on type parameters, Go lets us use functions in just the same way as any other kind of value. For example, functions can be passed as the parameters to other functions, or returned as results. The technical way to say this is that Go has *first-class functions*.

However, before generics, the uses of first-class functions in Go were rather limited by the need to always specify the type of their parameters and results. That meant that it was difficult or impossible to use some of the most desirable features of first-class functions available in other languages.

One common example of such a feature is *mapping* some function over a collection of elements. That is, applying the function to each element, with the final result being the transformed collection.

For example, if we wanted to double every integer in a slice, we could write some function `double` that doubles its input, and "map" that function over the slice. The result would be a slice where each element is twice the value of the corresponding element in the original slice.

This is nothing to do with Go's `map` type, but it conveys the same idea of mapping one set of values to another; in this case, by applying some arbitrary function to each value. Let's see how to do that.

Map

We're saying that the `Map` function takes some *arbitrary* function as a parameter, along with a slice, let's say, and applies the function to each element, returning the resulting slice. What does that look like? Let's first of all work out what the type of the arbitrary function needs to be.

Well, because it transforms an element of type `E` to another element of type `E`, its signature would be `func(E) E`. Let's give this type a name, then, to make it easier to think about:

```
type mapFunc[E any] func(E) E
```

Now we can write a generic function that takes a `mapFunc[E]` and applies it to every element of a slice of E:

```
func Map[S ~[]E, E any](s S, f mapFunc[E]) S {
    result := make(S, len(s))
    for i := range s {
        result[i] = f(s[i])
    }
    return result
}
```

Note that, like the `Identity` function in a previous chapter, `Map` takes two type parameters: the slice type *and* the element type. That's because, like `Identity`, it needs to return a result of the same type as whatever it was passed.

As usual, the implementation is not the interesting part. What's interesting is that now we can easily map any suitable function over a slice. For example, let's map the standard library function `strings.ToUpper` over a slice of strings:

```
s := []string{"a", "b", "c"}
fmt.Println(Map(s, strings.ToUpper))
// Output:
// [A B C]
```

Pretty neat! It's not like we couldn't have written an explicit loop here, of course, but this is a more concise and elegant way of expressing the same idea.

Type inference

What can we get away with here? Just for fun, let's try to map the `strings.ToUpper` function over a slice of some totally inappropriate type, such as `int`.

It turns out that, no, we can't sneak this one past the ever-watchful compiler:

```
s := []int{1, 2, 3}
fmt.Println(Map(s, strings.ToUpper))
// type func(s string) string of strings.ToUpper does not
// match inferred type mapFunc[int] for mapFunc[E]
```

We knew this wouldn't work, naturally, but often the best way to learn about something is to use it wrong on purpose, and see what happens. In this case, the error message is

worth parsing carefully, because it shows how Go does *type inference*: working out what E is in this instance, without us having to explicitly specify it.

Recall that `Map` declares a parameter of type `[]E`, and in this case that's `[]int`. Fine: Go can correctly infer that E is `int` here. But now there's a problem: the second parameter is a `mapFunc[E]`, for whatever E is. And since we know E is `int`, then the type of the function argument must be `func(int) int`.

The signature of the one we actually *passed*, though, is `func(string) string`. That's a straightforward type mismatch, hence the error.

Exercise: Func to funky

For the funcmap exercise, you'll be building a simple *dynamic dispatch* system in Go. That sounds fancy, but in plain language, it's a way of storing and calling a number of different functions by *name*.

We always call functions by name in some sense, of course, but we usually have to give the name explicitly in our program. Instead, your dynamic dispatcher will decide which function to call based on the value of some string that's not known until run-time.

Here are the tests you'll need to pass:

```
func TestFuncMap_AppliesDoubleTo2AndReturns4(t *testing.T) {
    t.Parallel()
    fm := funcmap.FuncMap[int, int]{
        "double": func(i int) int {
            return i * 2
        },
        "addOne": func(i int) int {
            return i + 1
        },
    }
    want := 4
    got := fm.Apply("double", 2)
    if want != got {
        t.Errorf("double(2): want %d, got %d", want, got)
    }
}

func TestFuncMap_AppliesUpperToUppercaseInput(t *testing.T) {
    t.Parallel()
    fm := funcmap.FuncMap[rune, bool]{
```

```
        "upper": unicode.IsUpper,
        "lower": unicode.IsLower,
    }
    got := fm.Apply("upper", 'A')
    if !got {
        t.Fatal("upper('A'): want true, got false")
    }
}
```

(Listing exercises/funcmap)

GOAL: Get these tests passing!

HINT: Your first job here is to create a `FuncMap` type which can store a number of named functions. That is, it's a map of `string` to some arbitrary function type.

For example:

```
fm := funcmap.FuncMap[int, int]{
    "double": func(i int) int {
        return i * 2
    },
    "addOne": func(i int) int {
        return i + 1
    },
}
```

To make things easier, let's limit ourselves to functions that take a single value of some arbitrary type T, and return a single result of some other type U. Can you figure out the necessary generic type definition?

You'll also need to implement an `Apply` method on `FuncMap` which will apply the specified function to some value and return the result. For example:

```
fmt.Println(fm.Apply("double", 2))
// Output:
// 4
```

Of course, there's no such type as plain `FuncMap` to write a method on, as we've seen. Instead, you'll need to write the method on a `FuncMap[T, U]`. Once you've established this, the signature of that method is pretty straightforward.

SOLUTION: Here's one possible answer. As usual, it's pretty concise, but might need some careful parsing to make sense of:

```
type FuncMap[T, U any] map[string]func(T) U

func (fm FuncMap[T, U]) Apply(name string, val T) U {
    return fm[name](val)
}
```

(Listing `solutions/funcmap`)

Filtering and reduction

Let's look at another popular functional pattern on slices: *filtering*.

`Filter`

`Filter` is the conventional name for a function that, like `Map`, takes an arbitrary function and applies it to each element of a slice. However, where `Map` uses the supplied function to *transform* each element, `Filter` uses it to decide which elements to keep and which to discard. The result of `Filter` is a slice containing only the elements that satisfied the `keep` function.

In other words, `Filter` takes an arbitrary slice, and also an arbitrary function that decides which elements of the slice to keep, and which to discard. Let's see if we can figure out how to write both functions.

What can we deduce about the signature of this `keep` function, then? Its job is to take some slice element, of type E, and return `true` if the element should be included in the slice of results, or `false` if it should be discarded.

```
type keepFunc[E any] func(E) bool
```

Fine. So now we can write `Filter` in terms of this `keepFunc` type:

```
func Filter[S ~[]E, E any](s S, f keepFunc[E]) S {
    result := S{}
    for _, v := range s {
        if f(v) {
            result = append(result, v)
        }
    }
```

```
    return result
}
```

Again, there's nothing very complicated about the implementation here. We range over the input slice checking whether `f` is true for each element. If so, we append it to the result slice. At the end, we have the slice containing only the elements of `s` for which `f` is true.

Filter functions

What kind of `keepFunc` could we use in real programs, then? Suppose we want to filter a slice of integers to find only the even values, for example.

In this case, our `keepFunc` would need to take an integer, and return `true` if it's even. Well, that doesn't sound too hard:

```
func(v int) bool {
    return v%2 == 0
}
```

Let's try it out, then,by passing this function literal to `Filter`:

```
s := []int{1, 2, 3, 4}
fmt.Println(Filter(s, func(v int) bool {
    return v%2 == 0
}))
// Output:
// [2 4]
```

Reassuring! This looks more or less the same as when we used `Map`, except that instead of passing an existing function `strings.ToUpper`, we supplied an anonymous *function literal* instead, which is quite idiomatic in Go.

Generic filter functions

We're not limited to passing only function literals to `Filter`, of course. We could certainly pass some existing *named* function, as we did with `strings.ToUpper`, too.

But there's another interesting option. What if we wanted to pass a *generic* `keepFunc`? That is, a function parameterised on some type T?

Could we do that? We might start by trying something like this:

```
func IsEven[T any](v T) bool {
    return v%2 == 0
}
```

But, thinking about it, we already know this can't work when T is constrained by `any`. Why not? Because we can only use the == operator with `comparable` types.

And Go's `%` (remainder) operator only works with integers, which is an even smaller type set. So we'll need to use `constraints.Integer` to ensure we only get values that support the `%` operator:

```
func IsEven[T constraints.Integer](v T) bool {
    return v%2 == 0
}
```

Let's try it out:

```
fmt.Println(Filter(s, IsEven[int]))
// Output:
// [2 4]
```

Wonderful!

Reduce

`Map` and `Filter` make perfect partners, but they're even better when united with their best friend, `Reduce`. Where `Map` uses a function to transform each element of a slice, and `Filter` uses a function to pick only the desired elements, `Reduce` uses a function to *combine* the elements of a slice.

This operation is sometimes also called "fold", "inject", or a variety of other names depending on the programming language. Essentially, it's about taking a sequence of values, doing some computation on them, and producing a single value as the result. For example, if we had a slice of numbers, we could add them up, producing a single result: the total.

In general, then, the function we pass to `Reduce` needs to take two things: a value representing the "current result" of the reduction (so far), and the next slice element to combine with it. It should return the updated current result, which in turn will be passed to the next invocation of the function, and so on, until we have the final result.

Let's try to work out the signature of such a function first:

```
type reduceFunc[E any] func(cur, next E) E
```

In other words, a `reduceFunc[E]` takes two parameters of type E, and returns a single E result.

Implementing `Reduce`

This makes sense, so now let's try to write `Reduce`:

```
func Reduce[E any](s []E, init E, f reduceFunc[E]) E {
    cur := init
    for _, v := range s {
        cur = f(cur, v)
    }
    return cur
}
```

The first parameter is the slice to operate on. The second is some starting value for the reduction (for example, zero). And the third is the `reduceFunc` to apply.

The implementation is pretty straightforward. We set `cur` equal to the initialising value `init`. Then, for each slice element, we call the `reduceFunc` with the current value of `cur`, and the element. The result of the `reduceFunc` becomes the next `cur`.

Let's try summing a slice of integers, for example. Our `reduceFunc` is easy to write. It needs to take a `cur` and `next` value and return their sum:

```
func(cur, next int) int {
    return cur + next
}
```

So let's pass this function literal to `Reduce` and see what it does:

```
s := []int{1, 2, 3, 4}
sum := Reduce(s, 0, func(cur, next int) int {
    return cur + next
})
fmt.Println(sum)
// Output:
// 10
```

Other reduction operations

Let's try another reduction operation. What about multiplication, for example?

```
s := []int{1, 2, 3, 4}
p := Reduce(s, 1, func(cur, next int) int {
    return cur * next
})
fmt.Println(p)
// Output:
// 24
```

We're not restricted to just numerical operations, of course. Here's a reduction involving strings, which just concatenates them all together:

```
s := []string{"a", "b", "c"}
j := Reduce(s, "", func(c, n string) string {
    return c + n
})
fmt.Println(j)
// Output:
// abc
```

Other considerations

You might be wondering why it's okay to use the `any` constraint in our reduction function. We've seen in some previous examples that `any` is too broad a constraint for some functions, because operators like `+` are not defined on every possible type.

Does that apply here? Shouldn't we have to constrain the `Reduce` function to only the set of types that support the `+` operator, then?

Constraints

Well, let's think about what would happen if we tried to `Reduce` a slice of things that *don't* support that operator: a struct, for example. You can't add two structs together, no matter how hard you try. So it seems like this would break our program.

What kind of `reduceFunc` would we need to pass in this case? Well, it would have to take two parameters of the struct type, whatever it is (let's say it's `struct{}`, but it doesn't matter). That's okay, but then it would have to add them together using the `+` operator, and that's *not* okay:

```
func sumStructs(cur, next struct{}) struct{} {
    return cur + next
```

```
}
// invalid operation: operator + not defined on cur (variable
// of type struct{})
```

We just can't write such a function in the first place! So the problem of what would happen if we tried to pass it to `Reduce` doesn't arise.

Concurrency

We've used pretty simple-minded implementations of `Map`, `Filter`, and `Reduce` in this chapter, just to make it easy to see what's going on. In practice, though, it would be useful to make these implementations a bit smarter. For example, we could imagine adding concurrency. If we had large slices to deal with, and if the map, reduce, or filter operations on them were relatively expensive, making them concurrent would help a lot.

For example, suppose we had a slice of strings representing web URLs, and we wanted to filter them to extract only those URLs which respond with OK status. We can imagine using our existing `Filter` implementation on this slice with some function that uses `http.Get` to call the URL and check the response status. But each request would take a fair amount of time (by computer standards): a few hundred milliseconds, perhaps.

Suppose each request takes about 250ms to complete, and there are 1,000 URLs to check. Filtering them sequentially would thus take more than two minutes.

Since we can make many HTTP requests at once, it seems silly not to start *all* these requests concurrently, and then wait for the responses to trickle in. The total filter time would then be a little more than the time of the slowest response, so of the order of 250ms.

That's a pretty big speedup, and since all the tasks in this workload are independent of each other (making it what's known as *embarrassingly parallel*), the speedup is linear with $N. In other words, concurrent filtering of slice elements always takes about the same total time, no matter how large the slice.

Naturally, adding concurrency to something like `Filter` also adds a good deal of complication and extra code, but that's okay: if it can be generic, then we only need to write it once! Indeed, some third-party packages already provide a concurrent `Filter` and related functions.

Exercise: Compose yourself

Composing functions means applying them in a chain, so that each function operates on the result of the previous one.

For example, if we had two arbitrary functions `outer` and `inner`, then we could compose them directly by writing:

```
outer(inner())
```

Your job in this compose exercise is to write a function that will take two other functions and compose them in just this way. For example, suppose we have two functions, `double`, which returns twice its input, and `addOne`, which returns its input plus one. And suppose we want to apply *both* of these functions to, say, the number 1. What's the result?

Well, it depends on which order we apply the functions in, doesn't it? So, if we write:

```
fmt.Println(double(addOne(1))
// Output:
// 4
```

we're asking what you get when you apply `double` to the result of `addOne(1)`. On the other hand, if we write:

```
fmt.Println(addOne(double(1))
// Output:
// 3
```

we're asking what `addOne` returns if we pass it the result of `double(1)`. Fine, so how should we specify the order when we pass the functions to `Compose`? Let's just use the same order that Go does: last named, first applied. So these two statements should give the same result:

```
fmt.Println(addOne(double(1))
fmt.Println(compose.Compose(addOne, double, 1))
// Output (in both cases):
// 3
```

Here's a test that ensures `Compose` applies the functions in the correct order:

```
func TestComposeAppliesFuncsInReverseOrder(t *testing.T) {
    t.Parallel()
    want := 4
    got := compose.Compose(double, addOne, 1)
    if want != got {
        t.Errorf("want %d, got %d", want, got)
    }
}
```

(Listing exercises/compose)

If you get 3 instead, then your `Compose` function is applying its arguments in the wrong order.

Of course, both `double` and `addOne` have the same signature: they take and return `int`. To make the exercise a little more interesting, though, we'll say that the two functions can have different signatures. For example:

```
func TestComposeHandlesDifferentFuncSignatures(t *testing.T) {
    t.Parallel()
    input := "HeLlO, wOrLd"
    want := 12
    got := compose.Compose(utf8.RuneCountInString, strings.ToUpper, input)
    if want != got {
        t.Errorf("want %q, got %q", want, got)
    }
}
```

(Listing exercises/compose)

Here, the inner function takes a string and returns a string, but the outer function takes a string and returns an `int`. Not just any combination of signatures is allowed, though: the outer function must take the same type of value as the inner functions returns. That makes sense, since we want to *apply* the outer function to the result of the inner one.

GOAL: Get the tests passing!

HINT: Applying the functions is relatively easy, so the tricky bit is likely to be figuring out the right signature for `Compose` itself.

If you're having trouble with this, remember that, as we just saw, the `inner` function will need to take a parameter of the same type as the value, and its result type is arbitrary. The `outer` function will need to take a parameter of whatever type `inner` returns, but *its* result type is also arbitrary.

So, all told, there are *three* type parameters involved here. I'm being a *bit* unfair on you, perhaps, since it's unlikely you'll need to write a generic function with a signature as complicated as this in practice. But isn't it nice to feel that you could if you had to?

It might seem a bit daunting, but just work carefully and go step by step, applying what you've learned in this chapter.

SOLUTION: Well, here's what I came up with. See how it compares to your version.

```
func Compose[T, U, V any](f func(U) T, g func(V) U, v V) T {
    return f(g(v))
}
```

(Listing `solutions/compose`)

Yes, there are a lot of Ts, Us, and Vs. Sorry about that.

7. Containers

> *A set is a Many that allows itself to be thought of as a One.*
>
> —Georg Cantor, quoted in Rudy Rucker's "Infinity and the Mind"

In the previous chapter we looked at some interesting generic functions that we can write on slices (and, by extension, maps). Maps and slices are so useful and ubiquitous in Go precisely because they can be used to hold elements of any type.

Before the introduction of generics, though, we were more or less limited to just maps and slices, because it wasn't possible to write user-defined containers of arbitrary types.

Now that we can, what container types, or other generic data structures, would it be interesting to write?

Sets

A *set* is the sort of thing we'd often like to have available in Go programs. It's like a kind of compromise between a slice and a map: a set is a collection of elements, all of the same type, but they're not in any specific order, and the elements must be unique.

Maps as sets

There's no built-in or standard library set type in Go, so before generics we would often use a map type as a quick-and-dirty substitute for a set. For example, we might write something like this:

```
var validCategory = map[string]bool{
    "Autobiography":       true,
    "Large Print Romance": true,
    "Particle Physics":    true,
}
```

—For the Love of Go

We're defining a set of valid categories for a curiously eclectic bookstore. Notice that the *element* type of the map doesn't matter here, since we're only really interested in the keys, which are strings.

It's convenient to use `bool` values in our map, though, since it means we can use them in conditional expressions like this:

```
if validCategory[category] {
```

As you probably know, a map index expression like this returns the zero value if the given key is not in the map. Since the zero value for `bool` is `false`, this means the `if` condition is false if `category` is not in the "set".

As neat as this is, it feels a little hacky, so it's nice that we should now be able to build ourselves a proper `Set` type using generics. Let's try.

Operations on sets

What kind of things would we like to be able to do with a set? There are many possibilities, but perhaps the most basic are adding an element, and checking whether the set contains a given element.

In other words, we'd like to be able to write something like this:

```
s := NewSet[int]()
s.Add(1)
fmt.Println(s.Contains(1))
// Output:
// true
```

Let's get to work!

Designing the `Set` type

We need something to keep the actual data in. A map would make sense, since it gives us the "unique key" property for free.

And we know the map keys will need to be of arbitrary (but comparable) type, since they represent the items in our `Set`. What about the element type?

We could use `bool` again, but maybe `struct{}` is a better choice in this case, since it takes up no space. So suppose we wrote some type definition like this:

```
type Set[E comparable] map[E]struct{}
```

For some comparable type E, we're saying, a `Set[E]` is a map of E to empty struct. Basically, this is just the same sort of trick as we pulled with the `validCategories` map, except that we're no longer restricted to just strings: we can create a `Set` of any `comparable` type.

Since maps in Go need to be initialised before we can do much with them, let's also provide a constructor function `NewSet`:

```
func NewSet[E comparable]() Set[E] {
    return Set[E]{}
}
```

Building out the machinery

Now let's add some useful methods. We'll start with a way to add new elements to the set.

The `Add` method

An `Add` method for sets is fairly simple to design. It should take some value of whatever our element type E is, and store it in the set. We can do that by assigning to the map:

```
func (s Set[E]) Add(v E) {
    s[v] = struct{}{}
}
```

The empty struct literal always looks a little weird to me (too many braces!), but it's just what we need here. The only element type we can *have* in this map is `struct{}`, and the only literal `struct{}` there can be is `struct{}{}`. So that's what we assign to the key `v` in our map.

The `Contains` method

The next job is to write `Contains`. Given some value of E, it should return true if the value is in the set, or false otherwise.

You probably know that a map index expression in Go can give you two values, if you ask for them. The second, conventionally named `ok`, tells us whether the key was in fact in the map. And that's the information we want here:

```
func (s Set[E]) Contains(v E) bool {
    _, ok := s[v]
    return ok
}
```

Let's try out some simple set operations:

```
s := NewSet[string]()
s.Add("hello")
fmt.Println(s.Contains("hello"))
// Output:
// true
```

Great! This is already useful. But let's see what we can do to make it even better.

Initialising with multiple values

It seems as though it would be convenient to initialise a set with a bunch of elements supplied to `NewSet`, rather than having to create the set first and *then* add each element one by one. In other words, we'd like to write:

```
s := NewSet(1, 2, 3)
```

Not only does this save time, but now we don't need to instantiate `NewSet`, because Go can infer the type of E from the values we pass to it.

Let's modify `NewSet` to take any number of E values:

```
func NewSet[E comparable](vals ...E) Set[E] {
    s := Set[E]{}
    for _, v := range vals {
        s[v] = struct{}{}
    }
    return s
```

```
}
```

While we're at it, we'll make the same change to `Add`, so that we can add any number of new members in one go:

```
func (s Set[E]) Add(vals ...E) {
    for _, v := range vals {
        s[v] = struct{}{}
    }
}
```

We're ready to roll:

```
s := NewSet(true, true, true)
s.Add(true, true)
fmt.Println(s.Contains(false))
// Output:
// false
```

What else could we do?

Getting a set's members

It'll almost certainly be convenient to get all the members of the set as a slice. One of the more annoying limitations of using maps as sets in Go is that it's not straightforward to do this.

Let's make it straightforward!

```
func (s Set[E]) All() []E {
    result := make([]E, 0, len(s))
    for v := range s {
        result = append(result, v)
    }
    return result
}
```

Now we can write:

```
s := NewSet(1, 2, 3)
fmt.Println(s.All())
```

```
// Output:
// [1 2 3]
```

A `String` method

In fact, let's add a `String` method too, so that we can print the set in a nice way:

```
func (s Set[E]) String() string {
    return fmt.Sprintf("%v", s.All())
}
```

Now we don't need to call `All` to print out the set, and we can write simply:

```
fmt.Println(s)
// Output:
// [2 3 1]
```

Notice that the members came out in a different order this time, and that's okay!

It's part of the definition of a set that its members *aren't* in any special order, just like a map. And because we're using a map to store the elements, we get that behaviour for free.

Logic on sets

So far, so good, and this is already useful. But part of the power of sets as a mathematical tool is that we can ask questions about how different sets relate to one another. For example, which elements do a given pair of sets have in common? Or what is the set of elements that are in *either* set?

Let's see what useful set-manipulation methods we can add along these lines.

Union

We'll start by computing the *union* of two sets: that is, the set that contains all the elements of both sets. How could we implement that?

One way would be to create a new set, using the members of set 1, then add the members of set 2. Here's what that looks like:

```
func (s Set[E]) Union(s2 Set[E]) Set[E] {
    result := NewSet(s.All()...)
    result.Add(s2.All()...)
```

```
    return result
}
```

Making `NewSet` and `Add` variadic has paid off already, since it makes `Union` much easier to write, with no looping.

If this works, then we should be able to find the union of two small sets of integers:

```
s1 := NewSet(1, 2, 3)
s2 := NewSet(3, 4, 5)
fmt.Println(s1.Union(s2))
// Output:
// [1 2 3 4 5]
```

Excellent! As we'd expect, `Union` has not just added together the two sets, it's also removed any duplicates. We didn't have to write any code to do this: again, it came free with the map.

Intersection

The *intersection* of two sets is the set of members that they have in common. Let's write `Intersection`, then, just for completeness:

```
func (s Set[E]) Intersection(s2 Set[E]) Set[E] {
    result := NewSet[E]()
    for _, v := range s.All() {
        if s2.Contains(v) {
            result.Add(v)
        }
    }
    return result
}
```

In other words, for each member of `s`, we check whether it is also present in `s2`. If so, we add it to the result set.

Does it work? Here goes:

```
s1 := NewSet(1, 2, 3)
s2 := NewSet(3, 4, 5)
fmt.Println(s1.Intersection(s2))
// Output:
```

```
// [3]
```

Encouraging!

A real-life example

Let's try out our new `Set` type in a semi-realistic application: checking job applications against the required set of skills for a vacancy.

Since we've just written `Intersection`, this should be easy. We just need to define the set of required skills, and find its intersection with the *candidate's* skills.

If there are any members in this set, then the candidate has at least one of the required skills, so we can hire them.

For example, suppose there's a job available which requires either Go or Java skills, but I happen to know Go and Ruby. Would I qualify?

Let's find out:

```
jobSkills := NewSet("go", "java")
mySkills := NewSet("go", "ruby")
matches := jobSkills.Intersection(mySkills)
if len(matches) > 0 {
    fmt.Println("You're hired!")
}
// Output:
// You're hired!
```

If only it were that easy.

Exercise: Stack overflow

A *stack* is an abstract data type which stores items in a last-in-first-out way. The only thing you can do with a stack is "push" a new item onto it, or "pop" the top item off it.

Your task in the stack exercise is to define a generic `Stack` type that holds an ordered collection of values, of as broad a range of types as possible.

You'll need to write a `Push` method that can append any number of items to the stack, in order, and a `Len` method that returns the number of items in the stack.

For example:

```
func TestPushToEmptyStackGivesStackWithLength1(t *testing.T) {
    t.Parallel()
```

```
    s := stack.Stack[int]{}
    s.Push(0)
    if s.Len() != 1 {
        t.Fatalf("want stack length 1, got %d", s.Len())
    }
}
```

(Listing exercises/stack)

You're also asked to write a `Pop` method that retrieves (and removes) the last item from the stack. `Pop` should also return a second `ok` value indicating whether an item was actually popped. If the stack is empty, then `Pop` should return `false` for this second value, but `true` otherwise. For example:

```
func TestPushPushPopPopLeavesStackEmpty(t *testing.T) {
    t.Parallel()
    s := stack.Stack[string]{}
    s.Push("a", "b")
    if s.Len() != 2 {
        t.Fatal("Push didn't add all values to stack")
    }
    s.Pop()
    want := "a"
    got, ok := s.Pop()
    if !ok {
        t.Fatal("Pop returned not ok on non-empty stack")
    }
    if want != got {
        t.Errorf("want %q, got %q", want, got)
    }
}

func TestPopReturnsNotOKOnEmptyStack(t *testing.T) {
    t.Parallel()
    s := stack.Stack[int]{}
    _, ok := s.Pop()
    if ok {
        t.Fatal("Pop returned ok on empty stack")
    }
```

```
}
```

(Listing exercises/stack)

GOAL: Get the tests passing!

HINT: This isn't really so different from the `Set` that we already tackled. The order of elements *does* matter with a stack, so that should influence your choice of the underlying data structure. With that settled, all that remains is to add the required methods.

If you've chosen your data type well, `Push` and `Len` should be straightforward: Go can do all the heavy lifting for us here. `Pop` will need a bit more thought. We need to handle the "empty stack" case, and also take care of removing the last element from the stack. See what you can do!

SOLUTION: Compare your version with mine. If yours is better, let me know!

```
type Stack[E any] struct {
    data []E
}

func (s Stack[E]) Len() int {
    return len(s.data)
}

func (s *Stack[E]) Push(vals ...E) {
    s.data = append(s.data, vals...)
}

func (s *Stack[E]) Pop() (v E, ok bool) {
    if len(s.data) == 0 {
        return v, false
    }
    v = s.data[len(s.data)-1]
    s.data = s.data[:len(s.data)-1]
    return v, true
}
```

(Listing solutions/stack)

There's a lot more we could do to make this stack implementation more useful, of course. But that's kind of the point.

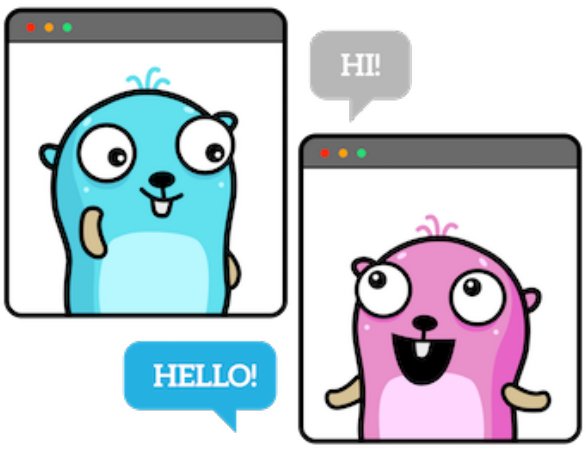
When we had to write, or generate, lots of duplicated code to implement data structures on multiple specific types, there was an obvious disincentive to put too much effort into it. Since it was impractical to provide general-purpose packages of such data structures, we tended to hack together our own versions as needed.

That being so, they generally weren't very sophisticated: no one wants to maintain N different versions of the same sophisticated data structure. So we often went without neat features like concurrency safety, high performance, state-of-the-art algorithms, and so on.

That's no longer the case, and we can write generic abstract data types that are as sophisticated as we like. Let's explore this a little in the next chapter, by trying to add a relatively simple feature to our new `Set` type: concurrency safety.

8. Concurrency

> *Any sufficiently complicated concurrent program in another language contains an ad hoc informally-specified bug-ridden slow implementation of half of Erlang.*
>
> —Robert Virding



Something that the built-in collection types in Go don't have is *concurrency safety*. That is to say, if one goroutine writes to a map, for example, and another goroutine reads it concurrently, then a *data race* exists, and something bad will happen.

Data races

At best, the program will terminate unexpectedly. At worst, it will continue to run, but with corrupted or incorrect data. It's up to you to avoid this situation.

The best way to prevent data races on shared data is *not to have any mutable shared data*, but this isn't always possible.

The second best way is *not to share the data*. That is, to allow only one *guard* goroutine to access it, and require other goroutines to issue read or write instructions to it via a *channel*, which is inherently concurrency-safe.

Mutexes

If neither of these options are available, and they certainly wouldn't be in a general-purpose set package, for example, then another possibility is to use a *mutex*.

A mutex (as in "mutual exclusion") is a concurrency-safe "lock" variable. Goroutines can use it to obtain "permission" before reading or writing the shared data, and release it afterwards.

Indeed, the standard library includes a `sync.Map` type, which is a concurrency-safe map with an internal mutex. Because it pre-dates generics, though, the only type of value you can store in it is `any`, which is annoying, so it's not much used.

There are several drawbacks to using mutexes in general. First, if goroutines often need concurrent access to the data, there will be *contention* for the mutex. In other words, goroutines will waste time waiting for the lock when they could have been doing useful work.

Deadlocks

Another problem with mutexes is that, when there is more than one, a *deadlock* can occur, bringing the program to a halt. For example, goroutine A holds mutex 1, and is waiting for mutex 2, while goroutine B holds mutex 2, and tries to get mutex 1.

Like two drivers at a road junction, each waiting for the other to go first, deadlocked goroutines will block each other forever. At best, a deadlock will cause your program to crash. At worst, it will *look* like it's okay, but stop doing any useful work, which can be hard to detect.

Third, since mutexes are "opt-in", people can use them wrongly. For example, they might forget to release the lock, which would be inefficient. Worse, they might forget to *obtain* it in the first place, which would be disastrous.

As a package author, you can make a mutex available, and add all the necessary locking logic to your code, but you can't actually ensure that people remember to *use* it.

Unless, of course, you force them to call your *accessor methods* to read or write the object, and that's the usual solution.

Let's see if we can build on the Set type we developed in the previous chapter to add concurrency safety using accessor methods in this way.

A concurrency-safe set type

We'll define a new type `SetC` which incorporates the same Set functionality we already have, but we'll add a mutex to it to make it safe for concurrent access.

Locking

We'll use the standard library `sync.RWMutex` type, which allows us to get either a *read lock* or a *write lock* on the data, depending what we need to do with it.

We need to keep both the mutex and the data map in the same value. That sounds like some kind of struct, doesn't it?

Let's start by writing something like this:

```
type SetC[E comparable] struct {
    mutex *sync.RWMutex
    data  map[E]struct{}
}
```

For some comparable type E, a `SetC[E]` is a struct containing a `*sync.RWMutex` field, and a `map[E]struct{}` field.

We need to store a *pointer* to the mutex, not the mutex value itself. Otherwise, if the struct is copied, we'll get two copies of the mutex, which would mean it no longer protects the data.

It never makes sense to copy a mutex, so when we store them in a struct, it's a good idea to use a pointer field to prevent any accidental copying.

The constructor

What would `NewSetC` need to do? Well, initialise the map and store any supplied values, just as before. It also needs to initialise the mutex:

```
func NewSetC[E comparable](vals ...E) *SetC[E] {
    s := SetC[E]{
        mutex: &sync.RWMutex{},
        data: map[E]struct{}{},
    }
    for _, v := range vals {
        s.data[v] = struct{}{}
    }
    return &s
}
```

A locking `Add` method

The behaviour of `Add` on our SetC type will be pretty much the same as before, with one important difference. It needs to lock the mutex before updating the map.

Specifically, it needs a write lock, to prevent any other goroutine from even *reading* the map while the update is happening.

Here's what that looks like:

```
func (s *SetC[E]) Add(vals ...E) {
    s.mutex.Lock()
    defer s.mutex.Unlock()
    for _, v := range vals {
        s.data[v] = struct{}{}
    }
}
```

The read-locking methods

What about `All`? That also needs to be lock-aware, but does it need a write lock? No, because it doesn't modify the data: it just reads it.

So a read lock is good enough here:

```
func (s SetC[E]) All() []E {
    s.mutex.RLock()
    defer s.mutex.RUnlock()
    result := make([]E, 0, len(s.data))
    for v := range s.data {
        result = append(result, v)
    }
    return result
}
```

`Contains` is also read-only, so we can use the same strategy:

```
func (s SetC[E]) Contains(v E) bool {
    s.mutex.RLock()
    defer s.mutex.RUnlock()
    _, ok := s.data[v]
    return ok
}
```

If we've done everything right so far, then we should be able to use a SetC in the ordinary, non-concurrent way that we did before. Let's try:

```
s := NewSetC('a', 'b')
s.Add('c')
fmt.Println(s.Contains('d'))
// Output:
// false
```

Testing concurrency safety

Great, but now we need to find out if the concurrency safety works. How could we do that?

We need to *use* our new type concurrently. That is to say, we need at least two goroutines. And to expose a data race if it exists, at least one of those goroutines will need to *write* to the SetC, while the other reads it.

Smoke tests

Just a single read and write might not be enough to trigger a crash; we might get lucky. So to increase the odds against us, let's read and write the SetC many times. Say, a thousand times.

If our locking works correctly, there should be no problem:

```
s := NewSetC(1, 2, 3)
var wg sync.WaitGroup
wg.Add(1)
go func() {
    for i := range 1000 {
        s.Add(i)
    }
    wg.Done()
}()
for i := range 1000 {
    _ = s.All()
}
wg.Wait()
fmt.Println("We made it!")
// Output:
// We made it!
```

This looks promising.

A fatal error

Great! But *was* our mutex really necessary, though? We can't tell from this result.

Maybe we would have been just fine without it. We wouldn't, but I don't expect you to take my word for that.

Let's try commenting out the locking code in `Add`. If we're right, this should cause a crash:

```
func (s *SetC[E]) Add(vals ...E) {
    // s.mutex.Lock()
    // defer s.mutex.Unlock()
    ...
```

Here we go:

```
fatal error: concurrent map iteration and map write

goroutine 1 [running]:
... [stack frames omitted]
main.(*SetC[...]).All(0xc0000b6000?)
        main.go:44 +0x145 fp=0xc0000c3e40
        sp=0xc0000c3d30 pc=0x108ba25
... [stack frames omitted]

goroutine 18 [runnable]:
main.(*SetC[...]).Add(...)
        main.go:36
... [stack frames omitted]
```

Yikes. We can see from this stack trace that the Go runtime detected a *read* on the map in `All` and a concurrent *write* in `Add`.

Since it can no longer guarantee the integrity of the data, it terminates the program immediately.

And that's exactly what we expected to happen when we disabled the mutex. Turns out the mutex was doing something useful after all!

The race detector

In fact, Go's built-in data race detector would have caught this problem for us, if we'd only thought to invoke it by adding the `-race` flag to our `go run` command. Let's see what it says now:

```
go run -race main.go

==================
WARNING: DATA RACE
Read at 0x00c000124180 by main goroutine:
  main.(*SetC[...]).All()
      main.go:43 +0xe4
  main.main()
      main.go:86 +0x5b2

Previous write at 0x00c000124180 by goroutine 7:
  runtime.mapassign_fast64()
      /usr/local/go/src/runtime/map_fast64.go:93 +0x0
  main.(*SetC[...]).Add()
      main.go:36 +0xc8
  main.main.func1()
      main.go:81 +0x4d
```

This underlines the importance of using the `-race` flag when testing any code involving concurrency, doesn't it? It's not infallible, but it goes a long way towards automatically catching data race situations like this.

If there *is* a data race in your program, it may not blow up today, and maybe not tomorrow. But it *will* blow up eventually, and most likely at a time of peak load, which is bound to make you somewhat unpopular.

And the rest

How should we update the other `Set` methods to work with `SetC`, such as `String`, `Union`, and `Intersection`?

We can copy the existing `String` method as is, because it only calls `All`, and that's already lock-aware.

The same goes for `Union`, which only calls `All` and `Add`.

We have a choice, though, when it comes to implementing `Intersection`. The old code will also be concurrency-safe without modification, because it also uses the lock-aware methods.

But let's take a closer look at the code for Set:

```
func (s Set[E]) Intersection(s2 Set[E]) Set[E] {
    result := NewSet[E]()
    for _, v := range s.All() {
        if s2.Contains(v) {
            result.Add(v)
        }
```

```
    }
    return result
}
```

There might be an efficiency issue here. Can you see what it is?

Since we call `s2.Contains` (which locks) for *every* member of `s`, there's potentially a lot of locking and unlocking going on.

Mutexes are fast, but they aren't instantaneous, so too much mutex *churn* hurts performance. To make `Intersection` efficient on large sets, it might be a good idea to eliminate the repeated calls to `Contains`.

Instead, we could get the read lock just once, execute the whole loop, and then release the lock.

Here's the result:

```
func (s SetC[E]) Intersection(s2 SetC[E]) *SetC[E] {
    result := NewSetC[E]()
    s2.mutex.RLock()
    defer s2.mutex.RUnlock()
    for _, v := range s.All() {
        _, ok := s2.data[v]
        if ok {
            result.Add(v)
        }
    }
    return result
}
```

It's a trade-off, though, isn't it? Locking the whole set while we read all the members avoids thrashing the mutex, but for large sets that might mean holding the lock for a long time.

So for some applications we might prefer more *granular* locking: that is, locking and unlocking the whole set for each individual read operation. It just depends.

Exercise: Channelling frustration

Can you put together everything you've learned in this chapter to solve the channel challenge?

It seems people have been using channels in Go a little too freely, and a scheme is now proposed to bill users by activity. Accordingly, you'll need to define a generic `Channel`

type with suitable instrumentation to track the number of sends and receives.

You have the following tasks to complete:

1. Define the `Channel` type.
2. Define a `New` constructor that returns an initialised `Channel` with a specified capacity, ready for use.
3. Define `Send` and `Receive` methods on the `Channel` to allow users to send and receive values of the appropriate type.
4. Define `Sends` and `Receives` methods that will report the number of sends and receives on the channel, for billing purposes.

Your new `Channel` type should act just like a regular Go channel: if you send something through it, it should arrive at the other end. For example:

```
func TestSendFollowedByReceiveGivesOriginalValue(t *testing.T) {
    t.Parallel()
    c := channel.New[int](1)
    want := 99
    c.Send(want)
    got := c.Receive()
    if want != got {
        t.Fatalf("want %d received, got %d", want, got)
    }
}
```

(Listing `exercises/channel`)

But the value your type will add is the ability to ask a channel how many sends and receives it has performed:

```
func TestSendsReturns3AfterThreeSendOperations(t *testing.T) {
    t.Parallel()
    c := channel.New[float64](3)
    c.Send(1.0)
    c.Send(2.0)
    c.Send(3.0)
    want := uint64(3)
    got := c.Sends()
    if want != got {
        t.Fatalf("want %d sends, got %d", want, got)
    }
}
```

```
func TestReceivesReturns2AfterTwoSendOperations(t *testing.T) {
    t.Parallel()
    c := channel.New[struct{}](1)
    c.Send(struct{}{})
    _ = c.Receive()
    c.Send(struct{}{})
    _ = c.Receive()
    want := uint64(2)
    got := c.Receives()
    if want != got {
        t.Fatalf("want %d receives, got %d", want, got)
    }
}
```

(Listing `exercises/channel`)

You'll need to ensure that your `Channel` is concurrency-safe, so make sure your solution passes the tests when the race detector is enabled. Use:

`go test -race`

In particular, you'll need to pass this demanding smoke test:

```
func TestChannelHandlesConcurrentSendsAndReceives(t *testing.T) {
    t.Parallel()
    c := channel.New[string](10)
    want := uint64(100)
    var wg sync.WaitGroup
    wg.Add(1)
    go func() {
        for i := uint64(0); i < want; i++ {
            c.Send("hello")
            _ = c.Receives()
        }
        wg.Done()
    }()
    for i := uint64(0); i < want; i++ {
        _ = c.Receive()
        _ = c.Sends()
```

```
    }
    wg.Wait()
    got := c.Sends()
    if want != got {
        t.Errorf("want %d sends, got %d", want, got)
    }
    got = c.Receives()
    if want != got {
        t.Errorf("want %d receives, got %d", want, got)
    }
}
```

(Listing `exercises/channel`)

GOAL: Get the tests passing!

HINT: You can use an ordinary Go channel internally, of course: no need to re-implement that behaviour. But you'll need to wrap the channel with some methods that track its send and receive activity.

It's easy to maintain a counter for sends and receives, and to increment that counter when the appropriate method is called. But can you do it in a concurrency-safe way? Let's find out.

SOLUTION: Here's a version that could do the job:

```
type Channel[T any] struct {
    lock            *sync.RWMutex
    ch              chan T
    sends, receives int
}

func New[T any](length int) *Channel[T] {
    return &Channel[T]{
        lock: &sync.RWMutex{},
        ch:   make(chan T, length),
    }
}
```

```
func (c *Channel[T]) Send(v T) {
    c.ch <- v
    c.lock.Lock()
    defer c.lock.Unlock()
    c.sends++
}

func (c *Channel[T]) Receive() T {
    v := <-c.ch
    c.lock.Lock()
    defer c.lock.Unlock()
    c.receives++
    return v
}

func (c *Channel[T]) Sends() int {
    c.lock.RLock()
    defer c.lock.RUnlock()
    return c.sends
}

func (c *Channel[T]) Receives() int {
    c.lock.RLock()
    defer c.lock.RUnlock()
    return c.receives
}
```

On reflection, though, something seems a bit off about this. Easily half the code is just about the paperwork of locking and unlocking the mutex. Can't we do better than this?

While a mutex is helpful for preventing concurrent access to mutable data in general, there's a shortcut for some specific, commonly-used types, such as integers: the `sync/atomic` package.

This means we can ensure *atomic* (that is, non-concurrent) operations on these types without having to constantly frob some mutex back and forth.

If we use `atomic.Uint64` counters for our channel stats, then, we can write a much cleaner-looking version:

```
type Channel[T any] struct {
```

```
    ch              chan T
    sends, receives atomic.Uint64
}

func New[T any](length int) *Channel[T] {
    return &Channel[T]{
        ch: make(chan T, length),
    }
}

func (c *Channel[T]) Send(v T) {
    c.ch <- v
    c.sends.Add(1)
}

func (c *Channel[T]) Receive() T {
    v := <-c.ch
    c.receives.Add(1)
    return v
}

func (c *Channel[T]) Sends() uint64 {
    return c.sends.Load()
}

func (c *Channel[T]) Receives() uint64 {
    return c.receives.Load()
}
```

(Listing `solutions/channel`)

Extending the Set type

Alas, space prevents us exploring all the possible enhancements we could make to the `Set` and `SetC` types, but here are a few ideas you might like to think about.

For one thing, it's not really very convenient to have two separate set types, each with exactly the same API but with different semantics. Ideally, we'd just have one `Set` type, but we could opt in to concurrency safety if we needed to. The set could use a flag inter-

nally to tell it whether or not it should use the lock.

How would that work? Perhaps we could have two constructors: `NewSet` and `NewSafeSet`, for example. Or, since it's always best to err on the side of safety, maybe `NewSet` and `NewUnsafeSet`. This way, users will get a safe set by default, and if they need an *unsafe* one for better performance, they can request it explicitly.

More set operations

There are lots more fun and useful things we could implement for our Set types. You might like to have a go at some of these yourself:

- A `Len` method
- A `Delete` method
- An `Equals` method
- An `IsSubset` method
- A `Subtract` method that removes all members that belong to some other set
- A `Difference` function to find the members that two sets don't have in common
- A `Make` function to pre-allocate memory
- `Map`, `Reduce`, and `Filter` operations on set elements

Also, the set types we've developed are *mutable*. That is, when you call `Add` on a set, for example, it updates the set in place.

For a more functional style of programming, we might like to have an *immutable* set type. In this case, methods like `Add` would *return* the modified set instead of changing the original.

Key-value stores and caches

Here's another idea. Right now, we can only store values in a set, but it would be nice to store *key-value pairs* instead. This would be a great way to *cache* data that's expensive to generate.

We already have almost all the machinery we need. Instead of always storing `struct{}` in the map, we could store some arbitrary value instead, couldn't we?

And since we tend to want to cache things in high-concurrency applications like network servers, we'll probably also want concurrency safety. So we could build a key-value store on top of a `SetC`.

Cached data usually has a certain *lifetime,* or period of validity, after which it *expires* and needs to be refreshed. So perhaps we could extend the key-value store to handle deleting, or refreshing, expired data.

Other container types

Sets are just the beginning. There are many other useful data structures we can now write generically in Go: trees, heaps, queues, rings, linked lists, graphs, vectors, matri-

ces, and so on.

Very likely we'll see lots of new packages providing high-quality implementations of these things for use in Go programs. That's great, and even if we don't often need to write generic code ourselves, we can still benefit from the general-purpose packages provided by others.

And if you feel like writing such a package yourself, go right ahead!

9. Packages

> *Somebody once asked me what language should be used to write a million-line program. I asked him how come he has to write a million-line program. Does he have a million-line problem?*
>
> —Charles Simonyi, quoted in Scott Rosenberg's "Dreaming in Code"

In the previous chapters, we've seen that the introduction of generics means that we can write general-purpose Go packages and modules in a new way.

Even more significantly, we can write new *kinds* of package: general-purpose data structures, and general-purpose functions on arbitrary collections of data.

Great, but what if we're not really interested in writing packages? What if we just want to sit back and enjoy using other people's packages? How will generics affect us in that case?

In this chapter, we'll discuss the new packages that accompany generics, and the changes that generics brings to the Go ecosystem in general.

The `cmp` package

The first, and smallest, new package is `cmp`, which we met in an earlier chapter. As we've seen, it defines a useful constraint, `Ordered`, which includes all built-in types that can be compared for relative magnitude using the > operator and friends, plus any types derived from them.

One other neat thing included in `cmp` is the `Or` function. This lets us give a list of values, and it will pick the first of them that is non-zero. For example:

```
x := cmp.Or(userX, "default X value")
```

If `userX` is the empty string, then `x` will be assigned the default we provided. Not a huge deal, but nice to have.

The `slices` package

We've been waiting for a `slices` package in Go for a long time, and it's finally here. We'll be using it a lot, so here are the most important things it provides.

Comparing slices

As you know, slice types in Go don't support the == operator, so `slices.Equal` now provides a standard way of checking whether two slices are equal:

```
s1 := []int{1, 2, 3}
s2 := []int{1, 2, 3}
fmt.Println(slices.Equal(s1, s2))
// Output:
// true
```

This is convenient when the element type happens to be `comparable`, but what if it's not? Or what if we want to compare elements in some special way?

In that case, we can use `slices.EqualFunc`, which lets us supply a suitable function to compare any two elements:

```
s1 := []string{"a", "b", "c"}
s2 := []string{"A", "B", "C"}
fmt.Println(slices.EqualFunc(s1, s2, strings.EqualFold))
// Output:
// true
```

An interesting property of `EqualFunc` is that it's defined on *two* slice types, not one, so we can use it to compare two different kinds of slices.

All we need to do is write our equality function to take *both* element types:

```
s1 := []int{0, 1, 2}
s2 := []rune{'a', 'b', 'c'}
equal := func(n int, r rune) bool {
    return n == int(r-'a')
}
fmt.Println(slices.EqualFunc(s1, s2, equal))
// Output:
// true
```

Sometimes it's not quite enough just to know whether two slices are *equal*: we might want to ask whether one is greater or lesser than the other, for example.

That's where `Compare` comes in. Given two slices, it returns 0 if they're equal, -1 if the first slice is lesser, or 1 if the first slice is greater.

```
smaller := []int{1, 2, 3}
bigger := []int{1, 2, 3, 4}
fmt.Println(slices.Compare(bigger, bigger))
// Output:
// 0
fmt.Println(slices.Compare(smaller, bigger))
// Output:
// -1
fmt.Println(slices.Compare(bigger, smaller))
// Output:
// 1
```

As you can see, a shorter slice is considered lesser by `Compare`, but what if the slices are the same length? In that case, the comparison depends on the first non-matching element:

```
s1 := []int{1, 2, 3}
s2 := []int{1, 2, 4}
fmt.Println(slices.Compare(s1, s2))
// Output:
// -1
```

And this is fine for element types that are constrained by `cmp.Ordered` (that is, they work with the > operator).

But what about other types, or what if finding their relative magnitude isn't so straightforward? For those, we can supply our own comparison function, using `CompareFunc`.

For example, let's compare two slices of integers in a special way that ignores their *sign*, and looks only at their absolute magnitude. In other words, these two slices should compare equal according to our scheme:

```
s1 := []int{-1, 2, -3}
s2 := []int{1, -2, 3}
```

We can write a suitable comparison function as follows:

```
func cmp(x, y int) int {
    absX := math.Abs(float64(x))
    absY := math.Abs(float64(y))
    if absX < absY {
        return -1
    }
    if absX > absY {
        return 1
    }
    return 0
}
```

Let's try it out:

```
fmt.Println(slices.CompareFunc(s1, s2, cmp))
// Output:
// 0
```

And just like `EqualFunc`, we can use `CompareFunc` with two different slice types, given a suitable comparison function.

Finding elements

Another common operation on slices is *finding* a specified element by index, or simply asking whether it's contained in the slice at all.

`slices.Index` will return the index of the first occurrence of a given element, or -1 if it's not found:

```
s := []int{1, 2, 3}
fmt.Println(slices.Index(s, 2))
// Output:
// 1
fmt.Println(slices.Index(s, 9))
// Output:
// -1
```

Maybe we don't want to search for a specific value, but instead we just want to know the first element for which some arbitrary function returns true. In that case, we can use `IndexFunc` instead of `Index`.

If we don't care about the index, but just want to know whether the element is anywhere in the slice, we can use `Contains`:

```
s := []int{1, 2, 3}
fmt.Println(slices.Contains(s, 1))
// Output:
// true
```

You might remember that we wrote this very function in a previous chapter. Well, now we don't have to! And if we need some customised way of checking for the element, we can use `slices.ContainsFunc` to supply a suitable function:

```
negative := func(n int) bool {
    return n < 0
}
s := []int{1, -2, 3}
if slices.ContainsFunc(s, negative) {
    fmt.Println("Don't be so negative!")
}
// Output:
// Don't be so negative!
```

Maxima and minima

Remember the generic `Greatest` function we wrote in an earlier chapter, to find the highest element of a given slice? Well, it turns out we don't need to write that function either, since it's part of `slices`:

```
s := []int{1, 3, 2}
fmt.Println(slices.Max(s))
// Output:
// 3
```

There's also `slices.Min`, for finding the lowest element, and, as you'd expect, we can supply custom comparison functions to `MaxFunc` or `MinFunc` if we need to.

Inserting and deleting

It's also perfectly natural to want to insert or delete elements of a slice, and accordingly the `slices` package provides `Insert` and `Delete` functions.

`Insert` inserts any number of elements at the specified index, returning the modified slice:

```
s := []string{"a", "c"}
s = slices.Insert(s, 1, "b")
fmt.Println(s)
// Output:
// [a b c]
```

Conversely, `Delete` deletes elements from the first index up to, but not including, the second index:

```
s := []string{"a", "b", "c"}
s = slices.Delete(s, 0, 2)
fmt.Println(s)
// Output:
// [c]
```

Cloning and compacting

To create a shallow copy of a slice, we can use `slices.Clone`:

```
s := []int{1, 2, 3}
fmt.Println(slices.Clone(s))
// Output:
// [1 2 3]
```

Or if we want to delete duplicate elements from the slice, like the Unix `uniq` command, we can use `slices.Compact`:

```
s := []int{1, 1, 1, 2, 3}
fmt.Println(slices.Compact(s))
// Output:
// [1 2 3]
```

Note that `Compact` only removes *successive* duplicate elements (just like `uniq`). If the duplicate elements aren't next to each other, they won't be removed:

```
s := []int{1, 2, 1, 3, 1}
fmt.Println(slices.Compact(s))
// Output:
// [1 2 1 3 1]
```

Again, if we want to override the default == comparison for identical elements, we can supply our own function to `CompactFunc`:

```
s := []string{"a", "A"}
fmt.Println(slices.CompactFunc(s, strings.EqualFold))
// Output:
// [a]
```

Growing and shrinking

As you probably know, slices in Go have some *length*, reported by the built-in `len` function. The length of a slice is the number of elements it contains.

A slice also has a *capacity*, which is reported by the `cap` function. You can think of the capacity of a slice as being the number of "slots" it has, even if not all those slots are currently occupied by elements.

A slice will automatically grow its capacity as needed when you add elements to it, which is convenient. But this process involves copying the slice and allocating a new chunk of memory every time it happens, which can result in a lot of unnecessary work.

For efficiency, we sometimes want to increase the capacity of a slice by a large amount in one shot. We can use `slices.Grow` to do this:

```
s := []int{1, 2}
fmt.Println(cap(s))
// Output:
// 2
```

```
s = slices.Grow(s, 10)
fmt.Println(cap(s))
// Output:
// 12
```

Conversely, if a slice is now a lot smaller than it used to be, and we'd like to reclaim that unused memory, we can use `slices.Clip` to reduce its capacity:

```
s := make([]int, 100)
fmt.Println(cap(s))
// Output:
// 100
s = slices.Delete(s, 0, 100)
s = slices.Clip(s)
fmt.Println(cap(s))
// Output:
// 0
```

Sorting

As we saw in the chapter on functions, the introduction of generics means we're no longer required to use the `sort` package to sort slices. Instead, we get a new family of sorting functions in the `slices` package.

For straightforward, in-place sorting of ordered elements, we can use `slices.Sort`:

```
s := []int{3, 1, 2}
slices.Sort(s)
fmt.Println(s)
// Output:
// [1 2 3]
```

If we combine this with `slices.Compact`, which we met earlier on, we can get the unique elements even from an initially unsorted slice:

```
s := []int{1, 2, 1, 3, 1}
slices.Sort(s)
fmt.Println(slices.Compact(s))
// Output:
// [1 2 3]
```

Maybe the elements are not one of the ordered types, though, or maybe we just want to customise the sorting for some other reason.

In this case, we can pass a comparison function to `SortFunc`, just as we did with `CompareFunc`:

```
s := []int{3, 1, 2}
slices.SortFunc(s, cmp.Compare)
fmt.Println(s)
// Output:
// [1 2 3]
```

The `cmp.Compare` function behaves exactly like `slices.Sort`, but we can write our own function if we need a special kind of comparison.

For a *stable* sort, where identical elements are guaranteed to remain in their original order, we can use `SortStableFunc`.

To detect if a slice is *already* sorted, use `IsSorted` or `IsSortedFunc`:

```
s := []int{1, 2, 3}
fmt.Println(slices.IsSorted(s))
// Output:
// true
```

Reversing and replacing

A common sort of technical interview question is to ask the candidate to write code to *reverse* the elements in a slice, and now the best answer to that is:

```
s := []int{1, 2, 3}
slices.Reverse(s)
fmt.Println(s)
// Output:
// [3 2 1]
```

We can also edit a specified portion of a slice using `Replace`. To do this, we pass in the slice, the start and end indexes of the elements to be replaced, and the new elements to replace them:

```
s := []string{"aardvark", "bear", "cat"}
s = slices.Replace(s, 1, 2, "bat", "bee", "bison")
fmt.Println(s)
```

```
// Output:
// [aardvark bat bee bison cat]
```

Searching

One benefit of having a sorted slice is that we can look for elements using a *binary search*. This is a common-sense way of looking for things that we all use with sorted data.

For example, imagine you're looking for a certain word in a dictionary. If you open the dictionary roughly in the middle, at some random word, then you immediately know which half of the dictionary your target word is in, don't you?

It either comes before the word you landed on, or after it. So you just cut your search space in half, with a single lookup. That's the "binary" part.

Let's say you now know that your target word is in the first half. So you can open the dictionary halfway through *that* section and see what word you land on. Repeat this "binary-chopping" process until you hit the target word.

This is obviously much faster than starting at page 1 and looking at every page in turn, especially if you're curious about the word "zyzzyva" (a kind of South American weevil, since you ask).

Given a sorted slice, then, we can use `slices.BinarySearch` to quickly find the index of a certain element. It returns two values: the index of the element, if found, and a `bool` indicating whether it *was* found.

```
s := []int{1, 2, 3}
fmt.Println(slices.BinarySearch(s, 2))
// Output:
// 1 true
```

Alternatively, we can use `BinarySearchFunc` to find the index of the first element for which some given function returns true.

If the element is *not* in the slice, then `BinarySearch` and `BinarySearchFunc` return the index where the element *would* be, if it were inserted at the correct place. For example:

```
s := []int{1, 2, 3}
fmt.Println(slices.BinarySearch(s, 999))
// Output:
// 3 false
s := []string{"a", "c"}
fmt.Println(slices.BinarySearch(s, "b"))
```

```
// Output:
// 1 false
```

In the first example, 999 was not found (hence `false`), but if it *were* to be in the slice, it would have to go at the end (at index 3). In the second example, the result is telling us that "b" should be inserted at index 1, in the middle of the slice.

The `maps` package

Like the `slices` package, `maps` is small but purposeful. Let's run through what's available in `maps`, with a few examples.

Comparing maps

We can compare two maps for equality, using `maps.Equal`:

```
m1 := map[string]bool{
    "generics": true,
}
m2 := map[string]bool{
    "generics": true,
}
fmt.Println(maps.Equal(m1, m2))
// Output:
// true
```

And to supply our own definition of "equality", or when using element types that don't work with ==, we can call `EqualFunc` instead.

Since map keys must be `comparable` anyway, there's no need to provide your own function to test *keys* for equality. They can just be compared with ==.

Deleting map entries

The built-in `delete` function already deletes a given key from a map, so the `maps` package doesn't need to duplicate that.

However, we can delete any map entries for which some function returns true, by passing it to `DeleteFunc`:

```
m := map[string]bool{
    "generics": true,
```

```
    "classes":  false,
}
findFalse := func(k string, v bool) bool {
    return !v
}
maps.DeleteFunc(m, findFalse)
fmt.Println(m)
// Output:
// map[generics:true]
```

Or if we just want to remove *all* entries from the map, we can call `maps.Clear`.

Cloning and copying

Just like with slices, we can use the `Clone` function to create a shallow copy of a map:

```
m1 := map[string]float64{
    "e": 0.5,
    "μ": 105.7,
    "τ": 1776.9,
}
m2 := maps.Clone(m1)
fmt.Println(maps.Equal(m1, m2))
// Output:
// true
```

There's no `Compact` function, since there can't be any duplicate keys in a map, but there *is* a `Copy` function that copies all the entries from one map to another.

If any of the copied keys already exists in the destination map, it will be overwritten:

```
m1 := map[int]bool{1: false, 2: true}
m2 := map[int]bool{1: true}
maps.Copy(m1, m2)
fmt.Println(m1)
// Output:
// map[1:true 2:true]
```

Note that the first argument to `Copy` is the destination map, and the second is the source. So we write, for example, `Copy(to, from)`, not `Copy(from, to)`.

New idioms

Now that we have these wonderful new packages, will they change the way we write idiomatic Go code? You bet.

Copying slices

For example, to create a copy of a slice, we used to have to write, laboriously:

```
b := make([]T, len(a))
copy(b, a)
```

Instead we can now write just:

```
b := slices.Clone(a)
```

Deleting slice elements

Similarly, to delete a number of consecutive elements from a slice, we used to have to write something like this:

```
s := []int{1, 2, 3, 4}
s = append(s[:1], s[3:]...)
fmt.Println(s)
// Output:
// [1 4]
```

This is much nicer:

```
s = slices.Delete(s, 1, 3)
```

Checking for slice elements

And we'll never have to write loops like this again:

```
s := []int{1, 2, 3, 4}
for _, v := range s {
    if v == 2 {
        fmt.Println("found it")
    }
}
```

Because it's just:

```
if slices.Contains(s, 2) {
    fmt.Println("found it")
}
```

And so on.

Exercise: Merging in turn

Over to you now: use the `map` package to solve the merge exercise.

This task is about writing a `Merge` function that takes any number of maps (all of the same arbitrary type) and merges them together, returning the result.

In other words, the result returned by `Merge` is a map containing all the key-value pairs in all its arguments.

For example:

```
m1 := map[int]bool{ 1: true }
m2 := map[int]bool{ 2: true }
fmt.Println(merge.Merge(m1, m2))
// Output:
// map[1:true 2:true]
```

If there are any conflicts (that is, if the maps have any key in common), then the "later" map wins. In other words, the maps are merged in the order that they're passed to `Merge`. For example:

```
func TestMergeCorrectlyMergesTwoMapsOfIntToBool(t *testing.T) {
    t.Parallel()
    inputs := []map[int]bool{
        {
            1: false,
            2: false,
            3: false,
        },
        {
            3: true,
            5: true,
        },
```

```
    }
    want := map[int]bool{
        1: false,
        2: false,
        3: true,
        5: true,
    }
    got := merge.Merge(inputs...)
    if !maps.Equal(want, got) {
        t.Errorf("want %v, got %v", want, got)
    }
}

func TestMergeCorrectlyMergesThreeMapsOfStringToAny(t *testing.T) {
    t.Parallel()
    inputs := []map[string]any{
        {
            "a": nil,
        },
        {
            "b": "hello, world",
            "c": 0,
        },
        {
            "a": 6 + 2i,
        },
    }
    want := map[string]any{
        "a": 6 + 2i,
        "b": "hello, world",
        "c": 0,
    }
    got := merge.Merge(inputs...)
    if !maps.Equal(want, got) {
        t.Errorf("want %v, got %v", want, got)
    }
}
```

(Listing exercises/merge)

Make sure that your `Merge` function accepts not just map types, but types *derived* from a map type. For example:

```
func TestMergeHandlesDerivedTypes(t *testing.T) {
    t.Parallel()
    type menu map[int]string
    m1 := menu{1: "eggs"}
    m2 := menu{2: "beans"}
    want := menu{
        1: "eggs",
        2: "beans",
    }
    got := merge.Merge(m1, m2)
    if !maps.Equal(want, got) {
        t.Errorf("want %v, got %v", want, got)
    }
}
```

(Listing exercises/merge)

GOAL: Get the tests passing!

HINT: What we'd *like* here is an easy way to copy elements from one map to another, overwriting any duplicate elements. If nothing occurs to you, re-read the section of this chapter on the `maps` package. It may be helpful.

We also need to handle, not just straightforward map types, but user-defined types *derived* from some map type. Recall that we saw how to handle derived types in the earlier chapter on operations, with an `Identity` function that takes derived types such as `StringList`. Well, we can use a similar trick with maps here.

SOLUTION: I'm a big believer in taking things easy. Why write a bunch of code when you can let the standard library take the strain?

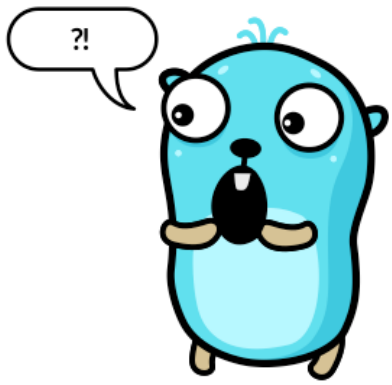
```
func Merge[M ~map[K]V, K comparable, V any](ms ...M) M {
    result := M{}
    for _, m := range ms {
        maps.Copy[map[K]V](result, m)
    }
```

```
    return result
}
```

(Listing solutions/merge)

10. Questions

> *Generics are an incredibly useful addition to the language that I'll almost never use.*
>
> —Scott Redig



We've covered a lot of ground already in this book, and I hope you've learned most of what you wanted to know about generics in Go, at least in theory.

In this chapter, then, we'll try to answer some of the most common questions about putting generics into *practice*.

Change

Things will be different now that Go has generics, of course, but in what ways, and by how much? And should you worry about it?

How much do I need to know about generics?

Not as much as some have feared!

The fact that you're reading this book implies that you're at least curious about the subject, but not everyone feels that way.

Some people didn't ask for generics, didn't want it, and just want generics to get off their lawn. How much new stuff will they still have to *learn,* though, even if they don't want to use generics themselves?

What you've read so far in this book is probably considerably more than any working Go programmer will actually *need* to know about generics. As we've seen, from the user's point of view, you can call generic functions or use generic types, in most cases, without even knowing that they *are* generic.

If you want to *write* generic functions and types, you'll need to know a little more, but not a lot. Knowing the type parameter syntax, and a few of the most commonly used constraints (`comparable`, `cmp.Ordered`), will be enough to get you started. And, as we've discussed, most people probably won't ever need to write generic functions or types anyway.

While you can become a competent software engineer by writing a lot of code, to become a master you need to *read* a lot of code. In order to understand the code you read, you need to be familiar with every aspect of Go syntax and usage, and that now includes generics.

So to some extent, to *know Go* requires knowing generics, even if it's not a language feature that you often need to use in practice. But that doesn't mean that you need to memorise every detail of, for example, the rules of type inference, or weird syntactic edge cases involving parsing ambiguity.

It's fun to learn about that postgraduate-level stuff, for some definition of "fun", but not actually necessary unless you're planning to write a Go compiler yourself.

Will generics drastically change the way I write Go?

Probably not.

There will, as we saw in the chapter on packages, be some important changes to the standard library that affect the majority of Go users.

For example, the `slices` and `maps` package introduce completely new APIs, albeit small ones. However, it's not essential for you to *use* them: they'll just save you some time and lines of code.

The other changes to the standard library will likely be rolled out gradually, incrementally, and in a backwards-compatible way.

So if you're not in a position to learn a whole bunch of new stuff just at the moment, that's okay. You'll be able to take advantage of the new or updated APIs in your own time and at your own pace.

And generics is a pretty niche thing anyway. No one should feel that just because generics are there, they're somehow expected to *use* them.

After all, we don't use goroutines in programs that don't need them, and indeed most programs don't. But when we *do* need them, it's great to have them available.

Do I need to change my existing code?

No.

The backwards compatibility promise means that no change to Go 1.x, including the introduction of generics, will break existing programs, with a few understandable exceptions, such as security fixes.

Should you change your existing code to use generics, though? That depends. If you currently *have* a problem that generics would solve, presumably you're currently solving it some other way.

If solving your problem with generics would bring enough benefit to justify the engineering work, then go ahead. Otherwise, if it isn't broken, you don't need to fix it.

Keep in mind also that the use of generics makes your program more complicated: it introduces more *abstraction*. That's not free.

> *Genericity introduces abstraction, and needless abstraction introduces complexity. Move cautiously!*
>
> —Robert Griesemer

Abstractions can be useful, of course, and software engineering is the *art* of creating useful abstractions. But they come at a cost to readability and simplicity.

We should only use any abstraction when the benefits outweigh the costs, and this applies equally to generics.

For example, there's not necessarily any point in replacing a function that takes an interface type like `io.Reader` with one that takes a type parameter.

Interfaces already solve this problem. There's no benefit from writing a generic function here to offset the extra complexity it introduces.

It *can* be worth replacing interfaces with generics in some situations. For example, if it gives you improved type safety, because you're now dealing with some specific type instead of an interface.

Performance

Similarly, if generics would allow you to make more efficient use of memory, or significantly improve performance, it's worthwhile. But making the language more complicated has an impact on the speed of the compiler, too. Let's unpack that.

What impact does generics have on performance?

There's more than one question here, actually, isn't there? One thing we could ask is "How much does the introduction of generics to Go affect the performance of programs *in general*?" Another is "How much does *using* generics in my program affect its performance?"

You might be surprised to learn that the answer to both questions is, in fact, "Not at all". Programs run no faster or slower when compiled with the latest version of Go, whether or not they use generics.

Maybe this shouldn't really be surprising, though, since we already know that generics is a purely compile-time phenomenon. Go *instantiates* every generic type or function call on some specific type. There are no generic types or functions in compiled Go programs, as we've discussed.

So while your binary may be fractionally *larger*, if it contains multiple compiled versions of some function, that binary will run no more slowly than if it had been written explicitly on a specific type.

However, that's not the end of the story. Your program may actually run *faster*, if you can use generics to eliminate reflection or interface values.

For example, the `tidwall/btree` project provides benchmarks suggesting that the version using generics is roughly twice as fast as the one using `any`.

This makes sense, since interfaces involve a run-time indirection that's bound to be slower than using concrete values directly. Projects that can use generics to eliminate the need for *reflection* may also see an even more significant speedup as a result.

Does generic code compile more slowly?

Another important question is how generics affects *compile* performance. Go was designed with speed of compilation very much in mind, especially for large codebases.

We'd expect generics to slow down compilation a tiny bit; after all, Go is *doing* more. But it's unlikely to make a significant difference:

> *There is certainly a very small extra amount of work to instantiate generic types and functions. But it should mostly be in the noise, similar to adding one extra function to a normal Go program; hardly noticeable in a full compile.*
>
> —Dan Scales

The vast majority of Go projects build extremely quickly anyway, of course, in just a few seconds, and in practice it's unlikely that most users will notice any difference at all.

So the answer to the performance question is that Go code with generics will perform faster, slower, or about the same, depending on exactly how you ask the question! That may not be a very satisfying answer, but it's at least an accurate one.

Downsides

Of course, nothing is free. Adding generics to Go makes the spec more complicated, and there's more for new Go programmers to learn (perhaps with the help of this book, though). So what are some of the negative aspects of this particular implementation of generics?

Why didn't they use angle brackets?

The question here is about the syntax for writing a list of type parameters. Here's what it looks like in Go, as you'll recall from previous chapters:

```
type Bunch[E any] []E
```

And the suggestion, or complaint, is that it should be instead:

```
type Bunch<E any> []E
```

In other words, why isn't the list of type parameters delimited with angle brackets (<>) instead of square brackets ([])? This is the syntax used by some languages including C++ and Java.

So why doesn't Go use angle brackets? Are the Go team just idiots, because they didn't think to check what other languages already did? Or are they sadists, because they just want to make life harder for you?

In fact, the original Go generics proposal used parentheses (( )) for type parameter lists, for symmetry with function parameter and result lists. It's reasonable to argue that new Go features should be designed in a way that's familiar from other constructs in *Go*, not Java, or any other language.

The trouble with parentheses is they it can lead to code that's a little awkward to read:

```
func Foo(T, U any)(p T, q U) (T, U) {
```

Maybe this is slightly too many parentheses. It was decided, then, to use a different delimiter for type parameter lists, to help them stand out in code. Unfortunately, not every imaginable delimiter character is available: some are already used for other things in Go.

The compiler has to be able to parse Go code unambiguously, and using < and > here would be a problem, because they're already valid operators in Go expressions. For example, consider an assignment statement like this:

```
a, b = w < x, y > (z)
```

Is this a pair of expressions (`w < x` and `y > (z)`), or an instantiation of a generic function call `w<x, y>` passing the argument `z`?

There's no way to know, so using angle brackets for generics would lead to ambiguities that can't be resolved. See the type parameters proposal for more details about this.

So while angle brackets might have made Go generics a tiny bit more familiar to people who were already used to generics in C++ or Java, for whatever that's worth, the option just wasn't available.

By all means register your disgust on the internet ("Worst. Syntax. *Ever*."), but this is where we are. Square brackets are fine. You'll get used to them; maybe you have already.

Still no option types

One problem that generics doesn't solve, at least in current versions of Go, is that of *option types*, sometimes called "Maybe" types. That is, types that can either contain some value, or no value.

For example, in some languages you can write code something like this, if `x` were a value of an option type:

```
switch x {
case Some(x):
    fmt.Println("Got value", x)
case None:
    fmt.Println("No value for you")
}
```

We can't really do that in Go, though we can approximate it with pointer types, where `nil` represents "no value for you". Generics doesn't help here either, since even a generic type must be instantiated on something specific, and most Go types can't be `nil`.

The Go idiom in this situation, though, is for functions to return "something and error", where the error value tells you whether or not there's any data in the other value. You could think of the "something and error" pair as *itself* being a kind of option type. Obviously, this won't satisfy Hacker News comment warriors, but perhaps that's not a good thing to optimise for.

It seems unlikely that anything like full-fledged option types will come to Go, but you never know. In the meantime, there's nothing stopping us implementing our own struct-based option types, along the lines of the optional package.

And generics makes it a lot easier to write such packages, so perhaps we'll see more use of option-style types in Go.

Still no enums

Another thing that people often miss in Go is *enumerated types*, or "enums", which are types that can hold only one of a set of allowed values. For example, an enum variable representing a DNA letter might be able to take only the values A, C, G, or T.

It's not possible to do this using the built-in types and operators in Go, and the most common approximation to it is to define a number of constants representing the "al-

lowed" values. That way, at least it's clear to users what values they're supposed to restrict themselves to.

However, there's no way to actually prevent people assigning some non-allowed value, which would be impossible with "real" enums. Again, we can define a struct type and provide validating accessor methods to simulate enums in Go, but it's not entirely satisfactory.

No proper union types

An extension of this idea is the *union type*, or *tagged union*. This is like an enum, but rather than containing one of a set of allowed values, it contains a value of one of a set of allowed *types*. For example, a given `Animal` might be either a `Cow`, or a `Chicken` (but not both).

If you think about it, a Go pointer is a kind of union, isn't it? It can contain either a valid pointer, or `nil` (but not both).

Indeed, if we had unions in Go, we could use them to implement option types. We don't, though, and again, generics don't solve any of these problems, if they are problems.

No macros

Another thing that Go still doesn't have, or want, is *template metaprogramming*, or *macros*, which we might describe loosely as "executing code at compile time". While generics provides a very simple form of templating, as we've seen in this book, it's limited to instantiating type parameters.

There's no way with Go generics to do any kind of computation at compile time, or make decisions based on conditions. We can't choose between different implementations of a function (*specialization*), or affect the build process in any way other than with build tags.

While metaprogramming is a powerful tool, it also introduces a great deal of complexity, and makes it much harder to build large programs efficiently. These downsides suggest that a facility like template metaprogramming is very unlikely to ever be added to Go.

No parameterised methods

Something else that's missing from Go right now is the ability to write *generic methods*, as we saw in an earlier chapter. This has made a lot of people very angry, and has been widely regarded as a bad move.

However, Jaana Dogan has suggested a neat workaround using a pattern called facilitators.

No generic packages

Go doesn't have *generic packages* (that is, the ability to choose which Go package to import based on a type parameter). This facility seems useful at first glance, but generics just doesn't really mesh well with Go's concept of packages:

> *It's very awkward for a function in one instantiation of a package to return a type that requires a different instantiation of the same package. And there is no particular reason to think that the uses of generic types will break down neatly into packages. Sometimes they will, sometimes they won't.*
>
> —Type Parameters Proposal

No "insert cool tech here"

A few more things that Go generics doesn't either enable or provide. There are no *operator methods*: for example, we can't write some method on a struct type that lets us use Go operators like >, ==, or + with it. That would be cool, but it's too late to add this to Go now.

Since Go doesn't have inheritance, there's also no *covariance* or *contravariance* (and if you don't know what these are, then I doubt you'll miss them). There's no *currying* (partial instantiation) or *variadic type parameters*. And the same applies to many arcane and exotic generics-related features that exist in other languages. You name it, Go doesn't have it!

It's not that any of these things are bad in themselves. Features are nice, but a language that has more features doesn't *necessarily* make us more productive, or result in better software. In fact, it can make it harder to learn that language in the first place.

There's a sweet spot for programming languages where they're powerful enough to solve any problem fairly efficiently, but still small and simple enough to be easy to learn. For most of its users, Go hits that spot.

More change

Go has, historically, been a fairly conservative language, in the sense that its design was fixed pretty early on, and hasn't changed much since. Proposals for changes that would break backward compatibility don't tend to be accepted. This isn't because the Go designers are lazy, or ignorant, or unwelcoming to new users. (They still *may* be all of those things, of course, but not because of this.)

Rather, it's that the Go team, and Go programmers everywhere, value consistency and stability above almost everything else. They'd rather not change the language than add some feature, however neat, just for the sake of it. Each tweak may be small, but they keep on coming. If you've tried to maintain important programs against an ever-shifting language or API, you'll appreciate the value of stability too.

On the other hand, some features really *are* worth it. We'll take a sneak peek at one such proposed new feature in the next chapter. In general, though, we don't expect Go to change significantly or often, and that's a good thing.

One consequence of this is that Go, even after a decade and a half, is still at version 1.x. So a lot of people have asked the obvious question:

Will there be a "Go 2"?

No.

> *There are no plans for anything called Go 2.*
>
> —Ian Lance Taylor

Since the first release of Go, people have had *opinions* about it. Boy, have they had opinions.

Most of them, of course, are to the effect that Go should be different in some way. These are usually framed as either "I hate Go because it doesn't have feature X", or alternatively "I love Go, but I wish it had feature X".

The vast majority of these suggestions have not been adopted, either because they weren't really actionable in themselves ("error handling sucks"), or because the benefits of adopting them wouldn't justify the costs.

So if you're considering proposing a change to Go, please don't, as a refusal often offends.

But even just reviewing and responding to such a never-ending stream of proposals is more than the Go team can keep up with, which is one reason why they've provided the language change proposal template.

Usually, in the process of filling out this paperwork, the proposer either gets bored or thinks better of their idea (or realises it's a slightly different form of something that's already been rejected). This seems both sensible and fair to me. If you're willing to put in the time to flesh out your proposal, rather than just saying "I want feature X", then the Go maintainers will put in the time and effort necessary to consider it.

There will be changes to Go

Some major changes, though, *have* been adopted, eventually (modules, error wrapping, generics). And there are several proposals that nearly made it, but despite well-considered designs and plenty of popular support (`try`), ultimately didn't *quite* clear the bar.

Most proposals, though, would break compatibility with existing code in one way or another, and so there's no point seriously discussing them. That doesn't mean that many of these proposals aren't excellent, or even that they can't or won't be adopted. They just won't be adopted in *Go*.

Accordingly, such proposals, rather than being rejected outright, are now tagged with the label "Go 2", meaning that they won't be adopted in Go 1, but are still worthy of consideration. This has helped mitigate criticism of the Go team for being stuck-in-the-mud, or "unfriendly" to newbies with bright ideas.

This was a good idea on the face of it, but also generated a widespread buzz that a "Go 2" might be imminent, which in turn prompted even *more* people to jump in with ingenious suggestions.

It won't be "Go 2"

Compounding this misunderstanding, the Go team even wrote a couple of blog posts giving the perhaps unfortunate impression that a version 2.x release of Go was in prospect. It's not, and never will be.

In general, successful programming languages don't release a version 2, for a very good reason. By definition (at least, by the definition of semantic versioning), this would be incompatible with code written for version 1. When this does happen (as it has with Python, for example), the effects are usually regrettable.

First, it renders most existing software useless, destroying the ecosystem that made the language successful. Changes significant enough to justify a "version 2" usually can't be backported to older code using automated tools.

And a successful language will have many users who are so heavily invested in it that they can't, or won't upgrade to the new version for many years, if ever.

Second, just because something is better doesn't mean it will automatically gain widespread adoption, or even any adoption (see IPv6 for details).

What the advent of version 2 *will* do is severely put off those who were considering adopting the language, at least until the situation has stabilised. So existing users quit, and the inflow of new users stops.

Third, the new version never completely displaces the old. So the upgrade fractures whatever language community remains, along with the software ecosystem.

Essentially, you end up with two different languages with the same name, and it would usually have been much better just to give the new language a new name.

But there will be a successor to Go (someday)

So what *will* happen with all those "Go 2" proposals?

One of two things is likely. If it's possible to water down or redesign the proposal in such a way that it could be introduced to Go in a non-breaking way, then that may well be done.

If not, one day some of these ideas may make it into a brand new language that, while clearly inspired by Go, will also be radically different from its predecessor.

If you're going to *make* breaking changes, you may as well go big or go home. This hypothetical future language may be as different from Go as Go is from, say, C.

And that's the point: this language won't *replace* Go, any more than Go has replaced C. It will take the best ideas from Go, combine them with the best proposed improvements, and incorporate the current state of the art in programming language design.

The result will be something just as unique and wonderful as Go. It just won't be Go 2.

11. Iterators

> *Nothing is ever so good that it can't stand a little revision.*
>
> —Rebecca Solnit, "Hope in the Dark: Untold Histories, Wild Possibilities"

As we saw in the previous chapter, Go doesn't change much, and that's a great thing. When changes do come, they almost always extend the existing language in a useful way, and generics is a good example of this. In this chapter, let's talk about another fairly new feature in Go (introduced in version 1.23): *iterators*, also known as "range over func".

So what's that all about, and how does it work?

Introduction to iterators

An iterator is a function that "yields" one result at a time, instead of computing a whole set of results and returning them all at once. When you write a `for` loop where the `range` expression is an iterator, the loop will be executed once for each value returned by the iterator.

That sounds complicated, but it's actually very simple. It's like a short-order cook who makes a dish for each customer on demand, as they come in, rather than cooking a whole day's worth of meals in advance.

Let's break it down, then. First of all, what's wrong with just returning a slice?

Why are iterators useful?

As you know, you can use `range` to loop over a bunch of different kinds of thing in Go: slices, maps, strings, channels. The most common case is probably looping over a slice.

This slice could be a local variable, or it could be the result of calling some function that returns a slice. For example:

```
for _, v := range Items() {
  fmt.Println("item", v)
}
```

We could write the `Items` function something like this:

```
type Item int

func Items() (items []Item) {
    ... // generate items
    return items
}
```

But there are some disadvantages to this way of doing it. First, we have to wait until the whole slice has been generated before we can start to process the first element of it.

Second, we have to allocate memory for the whole slice even though we're only using one element at a time. Third, if we don't end up using all the elements, then we've wasted time and memory computing stuff we didn't use. How annoying!

So what if there was something *like* a slice, but it produced just one element at a time, as needed? Let's call this handy little gadget an *iterator*, as in "thing you can iterate over".

In other words, a `for` loop over such an iterator would execute once for each result that the iterator produces—each `Item` in our example.

What is an iterator, then? How does it work under the hood?

The signature of an iterator function

An iterator is just a function, but with a particular signature. For example, an iterator of `Item` would be:

```
func (yield func(Item) bool)
```

As you can see, an iterator takes a function as a parameter, conventionally called `yield`. It's the function that the iterator will call to yield each successive value to the `range` loop.

Once the iterator has yielded all the items available, it returns, signalling "iteration complete".

For example, here's how we could write an iterator of `Item`:

```
func iterateItems(yield func(Item) bool) {
  items := []Item{1, 2, 3}
  for _, v := range items {
    if !yield(v) {
        return
    }
  }
}
```

Every time you want to produce a result, you pass it to `yield`, and control goes back to the loop statement, which would look like this:

```
for v := range iterateItems {
  ...
}
```

Assuming the loop keeps executing, `yield` will keep returning `true`, meaning "more values please". But if the loop ever terminates (maybe with `break` or `return`, for example), then `yield` will return `false`. That gives `Items` a chance to do any necessary cleanup.

Notice that if `yield` does return `false` ("stop iteration, the loop is done"), the iterator must detect this and bail out, rather than waste time computing more results that won't be needed. If it tries to yield again, the runtime will panic:

```
range function continued iteration after exit
```

Using iterators in programs

A function like `iterateItems` is simple enough for us to get the idea, but in real Go programs, we probably won't range directly over a function like this. Instead, the most useful way to create iterators is to return them from a function call.

Single-value iterators: `iter.Seq`

For example, suppose we rewrote `Items` so that, instead of returning a slice of `Item`, it returned an *iterator* of `Item`. How would we declare such a result type in our function signature?

We already know that the signature of an `Item` iterator is:

```
func (yield func(Item) bool)
```

But it would be annoying to write this out in full as the result type of a function like `Items`, so instead we can use the standard library `iter` package, which defines the type `Seq` for us.

Now it's easy to describe the result of `Items`, as simply `iter.Seq[Item]`:

```
import "iter"

func Items() iter.Seq[Item] {
  return func(yield func(Item) bool) {
    items := []Item{1, 2, 3}
    for _, v := range items {
      if !yield(v) {
          return
      }
    }
  }
}
```

Notice that `Items` is not an iterator *itself*. It doesn't have the required signature. Instead, it *returns* an iterator.

Since we only receive only one value in our `for .. range` statement, our loop now looks like this:

```
for v := range Items() {
  fmt.Println("item", v)
}
```

In other words, what we're ranging over is not some function, like `iterateItems`, but the result of *calling* a function: `Items()`. And that result is an iterator; specifically, an `iter.Seq[Item]`.

Two-value iterators: `iter.Seq2`

Notice that we only receive *one* value in our example `for ... range` statement, not two as in the "loop over slice" version. That value, straightforwardly enough, is the item itself: we don't need the index here.

But what if we *do* want the index? For example, we might want to print a numbered list of items. In this case, we can write exactly the same two-valued `for ... range` statement that we would with a slice:

```
for i, v := range Items() {
  fmt.Println("item" i, "is",v)
}
```

With our existing `Items` function, this won't work, because it only yields one value at a time:

```
range over Items() (value of type iter.Seq[Item]) permits only one iteration variable
```

What do we need to change about the iterator to make this work? Well, the signature of its `yield` function changes, since it must yield two items instead of one:

```
func (int, Item) bool
```

Makes sense, doesn't it? Each time round the loop, this `yield` function will produce another index-`Item` pair.

This is a different kind of iterator signature, so let's call it an `iter.Seq2[int, Item]`, because it yields two values. Here's how we could write that:

```
func Items() iter.Seq2[int, Item] {
  return func(yield func(int, Item) bool) {
    items := []Item{1, 2, 3}
    for i, v := range items {
      if !yield(i, v) {
          return
      }
    }
  }
}
```

Let's try our loop again to see if it works now:

```
for i, v := range Items() {
  fmt.Println("item" i, "is",v)
```

```
}
```

```
item 0 is 1
item 1 is 2
item 2 is 3
```

Neat! By the way, we don't *have* to receive that index when ranging over `Items`, even though it's a `Seq2`. If we didn't need the index in a particular loop, we could ignore it with the blank identifier `_`, just as we do when ranging over a slice.

And in loops where we don't care about either the index *or* the value, we don't have to receive anything at all. We can write just:

```
for range Items() {
  fmt.Println("this prints once for each item")
}
```

Dealing with errors

The iterators we've talked about so far are what we might call *infallible*: in other words, they'll always produce as many results as they're asked for, or as many as there are available.

A good example of this is something like iterating over a collection type. For example, suppose we had a `Set[E comparable]` type, of the kind we developed in a previous chapter. Instead of the `All` method returning a slice, it would be nice if we could return an iterator:

```
func (s *Set[E]) All() iter.Seq[E]
```

But what about when the iterator has to *do* something to generate each item, and that something could fail? For example, reading something from an `io.Reader` or a `bufio.Scanner`. We know this could produce an error, so what should we do with it?

Well, the iterator *could* just return in this case, since it has no more items to yield, but then we've lost the error. The calling loop has no way to know that the iterator ran into a problem and had to stop early.

Instead, let's take advantage of the fact that we can yield *two* results (if we're an `iter.Seq2`, that is). With a slice, these two things are usually the index and the value, but here they *need* not be. We could yield the value and an error instead, for example:

```
func Lines(file string) iter.Seq2[string, error]
```

When we successfully read a line, we can return it along with a `nil` error, just like the plain old Go functions we love so well. On the other hand, when there's an error, we

can use that `error` result to signal what it was.

That does mean that the calling loop will need to *check* the iterator's error, but this seems reasonable. It's no different to calling a regular function that returns a slice and `error`, but this way, we get the benefits of an iterator.

Cleaning up

We've seen that iterators need to check the result of calling their `yield` function, because that's how they know when to stop. Why does that matter? I mean, when the calling loop is done, why does `yield` need to return a `false` result? Why does it return a result at all, indeed?

The answer is that maybe the iterator uses some resources that need to be cleaned up. For example, it might be holding on to a database connection that's the source of the items.

In this case, we can treat a `false` result from `yield` as a signal: "Clean up, you're done".

Embracing iterators

Now that we know something about how iterators work, let's talk about what it means for us as programmers. How does this new feature change the way we write programs, and design APIs? Here are a few ideas.

Composing iterators

We've talked about functions that return iterators, but what about functions that *take* iterators, too? For example, we could write something like:

```
func PrintAll[V any](seq iter.Seq[V]) {
    for v := range seq {
        fmt.Println(v)
    }
}
```

Notice that we don't even care what `seq` is an iterator *of*. It doesn't matter. We can `range` over it regardless.

And if we can pass iterators to functions that return *other* iterators, then we can *compose* iterators:

```
func PrintPrimes() {
  for p := range Primes(Integers()) {
    fmt.Println(p)
```

```
  }
}

func Primes(seq iter.Seq[int]) iter.Seq[int] {
  return func(yield func(int) bool) {
    for n := range seq {
      if isPrime(n) {
        if !yield(n) {
          return
        }
      }
    }
  }
}

func Integers() iter.Seq[int] {
  return func(yield func(int) bool) {
    for i := range math.MaxInt {
      if !yield(i) {
        return
      }
    }
  }
}
```

Okay, it's a contrived example, but you get the point.

When iterators beat channels

Even before the advent of iterators, we could get a somewhat similar behaviour by calling a function that returns a *channel* of its results instead of a slice.

Looping over this channel with `range` behaves exactly the way you'd expect, which is to say just the same as with an iterator. But there are at least two things about this solution that aren't ideal. One is that your program is now *concurrent*, which means it's probably incorrect. (Prove me wrong, Gophers. Prove me wrong.)

Concurrent programming is just hard, and there are many easy-to-make mistakes that can cause a concurrent program to deadlock, crash, and so on. It's not a tool we deploy lightly, and it wouldn't be worth the extra complexity just to iterate over a slice.

The second problem is about what happens if the loop exits early. If you stop listen-

ing to a channel, whoever's trying to send to it will just block (in the case of a buffered channel, when the buffer fills up). In other words, the sending goroutine will *leak*: it just hangs around, taking up memory, and it has no way to ever exit. In a long-running program, leaks are something to avoid.

Sure, we *do* have ways to deal with that, like passing contexts that can be cancelled, and so on. And channels are very useful in all sorts of other situations. I'm not saying the problems with using channels in this way are *insoluble*, just that they're annoying, and iterators do this particular job in a much more user-friendly way.

On the other hand, if your program *is* concurrent, using channels rather than iterators probably makes sense in most contexts.

Standard library changes

Now that iterators are part of Go, the standard library has acquired some new APIs in order to take advantage of the new feature. It makes sense that the existing `slices` and `maps` packages should be able to return iterators, since they're about collection types, and indeed that's the case.

Slices

The new `slices.All` and `slices.Values` functions return iterators of index-value pairs, and values, respectively:

```
s := []int{1, 2, 3}
for i, v := range slices.All(s) {
  fmt.Println("item", i, "is", v)
}
// Output:
// item 0 is 1
// item 1 is 2
// item 2 is 3
for v := range slices.Values(s) {
  fmt.Println(v)
}
// Output:
// 1
// 2
// 3
```

Since we often want to deal with the results of an iterator *as* a slice, the new `slices.Collect` function provides an easy way to do that:

```
s := slices.Collect(slices.Values([]int{1, 2, 3}))
fmt.Println(s)
// Output:
// [1 2 3]
```

Maps

Similarly, we now have `maps.All`, `maps.Keys`, and `maps.Values` for iterating over maps:

```
m := map[string]float64{
  "ε": 8.854187817620389e-12,
  "π":  math.Pi,
  "e":  math.E,
  "ħ":  1.054571817e-34,
}
for k, v := range maps.All(m) {
  fmt.Println("constant", k, "is", v)
}
// Unordered output:
// constant e is 2.718281828459045
// constant ħ is 1.054571817e-34
// constant ε is 8.854187817620389e-12
// constant π is 3.141592653589793
for k := range maps.Keys(m) {
  fmt.Println(k)
}
// Unordered output:
// π
// e
// ħ
// ε
for v := range maps.Values(m) {
  fmt.Println(v)
}
// Unordered output:
// 3.141592653589793
// 2.718281828459045
// 1.054571817e-34
```

```
// 8.854187817620389e-12
```

About this book

Who wrote this?

John Arundel is a Go teacher and mentor of many years experience. He's helped literally thousands of people to learn Go, with friendly, supportive, professional mentoring, and he can help you too. Find out more:

- Learn Go remotely with me

Feedback

If you enjoyed this book, let me know! Email go@bitfieldconsulting.com with your comments. If you didn't enjoy it, or found a problem, I'd like to hear that too. All your feedback will go to improving the book.

Also, please tell your friends, or post about the book on social media. I'm not a global mega-corporation, and I don't have a publisher or a marketing budget: I write and produce these books myself, at home, in my spare time. I'm not doing this for the money: I'm doing it so that I can help bring the power of Go to as many people as possible.

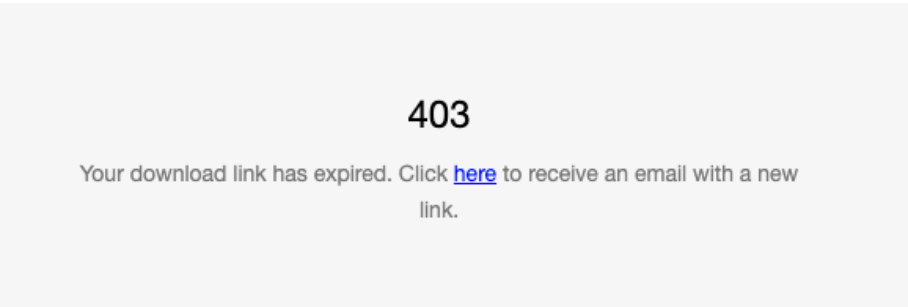
That's where you can help, too. If you love Go, tell a friend about this book!

Free updates to future editions

All my books come with free updates to future editions, for life. Make sure you save the email containing your download link that you got when you made your purchase. It's your key to downloading future updates.

When I publish a new edition, you'll get an email to let you know about it (make sure you're subscribed to my mailing list). When that happens, re-visit the link in your original download email, and you'll be able to download the new version.

NOTE: The Squarespace store makes this update process confusing for some readers, and I heartily sympathise with them. Your original download link is only valid for 24 hours, and if you re-visit it after that time, you'll see what *looks* like an error page:



But, in tiny print, you'll also see a "click here" link. Click it, and a new email with a new download link will be sent to you.

Join my Go Club

I have a mailing list where you can subscribe to my latest articles, book news, and special offers. I'd be honoured if you'd like to be a member. Needless to say, I'll never share your data with anyone, and you can unsubscribe at any time.

- Join Go Club

For the Love of Go

For the Love of Go is a book introducing the Go programming language, suitable for complete beginners, as well as those with experience programming in other languages.

If you've used Go before but feel somehow you skipped something important, this book will build your confidence in the fundamentals. Take your first steps toward mastery with this fun, readable, and easy-to-follow guide.

Throughout the book we'll be working together to develop a fun and useful project in Go: an online bookstore called Happy Fun Books. You'll learn how to use Go to store data about real-world objects such as books, how to write code to manage and modify that data, and how to build useful and effective programs around it.

The Power of Go: Tools

Are you ready to unlock the power of Go, master obviousness-oriented programming, and learn the secrets of Zen mountaineering? Then you're ready for The Power of Go: Tools.

It's the next step on your software engineering journey, explaining how to write simple, powerful, idiomatic, and even beautiful programs in Go.

This friendly, supportive, yet challenging book will show you how master software engineers think, and guide you through the process of designing production-ready command-line tools in Go step by step.

Further reading

You can find more of my books on Go here:

- Go books by John Arundel

You can find more Go tutorials and exercises here:

- Go tutorials from Bitfield

I have a YouTube channel where I post occasional videos on Go, and there are also some curated playlists of what I judge to be the very best Go talks and tutorials available, here:

- Bitfield Consulting on YouTube

Credits

Gopher images by the magnificent egonelbre and MariaLetta.

Acknowledgements

My thanks are due to my students at the Bitfield Institute of Technology who unstintingly gave of their time to help review the drafts of this book, discuss its ideas, provide feedback, and suggest questions and examples. I'm especially grateful to Brian Hasden, who came up with the "cow and chicken" example.

Many thanks also to the hundreds of beta readers who willingly read early drafts of various editions of this book, and took the time to give me detailed and useful feedback on where it could be improved. Shiv Patil, Andrew Smith, BK Lau, Dan Macklin, Pedro Sandoval, and Pavel Anni in particular helped me a good deal.